TechCart

Canlı Yayına Hazırlık ve 1000 Eşzamanlı Kullanıcıya Doğru Teknik Rehber

Türkçe mühendislik ve uygulama rehberi | 22 Ekim 2026 | Sürüm 1.0

Bu rehber, TechCart'ın bugün yapabildiklerini, hangi özelliklerin henüz hedef olduğunu ve bu hedeflere hangi gerekçelerle ilerleyeceğimizi açıklar. Amaç, çalışan bir alışveriş uygulamasını doğruluğu sınanmış, davranışı ölçülebilen ve sorun çıktığında işletilebilen bir ürüne dönüştürmektir.

Ana hedef 1000 eşzamanlı aktif kullanıcıya doğru ilerlemektir. Bu bir kapasite sonucu değildir. İlk yayın kapısı, 50 eşzamanlı kullanıcıyla yapılan kabul testidir. İlk yayın herkese açık bir demo olacak; ödeme sağlayıcısında iyzico sandbox kullanılacaktır.

Bu belge hazırlanırken uygulama kodu, veritabanı ve çalışan servisler değiştirilmez; yük testi çalıştırılmaz. Kod inceleme tarihi ile önceki çalıştırmalardan elde edilen kanıtlar ayrı belirtilir. Örneklerde “hedef uygulama” yazması, ilgili yapının henüz projede bulunmadığını gösterir.

İçindekiler

1. Bugün neredeyiz, neyi kanıtladık? . . . 3
2. Koruyacağımız mimari ve bir alışverişin yolculuğu . . . 4
3. Ortamlar, kurulum ve konfigürasyonun anlamı . . . 6
4. Siparişin tamamını tek işlem olarak düşünmek . . . 7
5. Stok rezervasyonu ve eşzamanlılık . . . 8
6. Ödeme, belirsizlik ve güvenli tekrar . . . 9
7. Kaybolmayan olaylar: outbox ve inbox . . . 11
8. Tekrar, sıra ve hata kuyruğu yönetimi . . . 12
9. Log, metrik ve trace ile sistemi anlamak . . . 13
10. Gözlemleme altyapısı, paneller ve maliyet . . . 14
11. PostgreSQL performansını doğru ölçmek . . . 16
12. Redis cache: nerede, neden ve nasıl? . . . 17
13. Elasticsearch ve frontend istek davranışı . . . 18
14. 1000 kullanıcı hedefini doğru tanımlamak . . . 19
15. Sentetik veri, oturumlar ve ödeme simülasyonu . . . 20
16. Yük testlerinin sırası ve uygulanması . . . 22
17. k6 ile ölçülebilir senaryo yazmak . . . 24
18. Hata deneyleri ve doğruluk testleri . . . 27
19. Raporlama ve kapasite kararları . . . 29
20. İnternete güvenli açılmak . . . 31
21. Otomatik doğrulama, dağıtım ve geri dönüş . . . 32
22. Yedek, geri yükleme ve işletim . . . 33
23. Bütçe, yol haritası ve yayın kapıları . . . 34
24. Karar özeti ve sık sorulan sorular . . . 36
25. Terimler sözlüğü . . . 38
26. Kaynaklar ve belge doğrulama notları . . . 40

1. Bugün neredeyiz, neyi kanıtladık?

Mevcut olmak ile doğrulanmış olmak farklıdır

Bir özelliğin kodda bulunması, bütün koşullarda doğru çalıştığının kanıtı değildir. Örneğin stok azaltan bir SQL sorgusu kaynak kodda görülebilir. Bu, aynı ürüne 100 kişi yöneldiğinde bütün siparişlerin doğru sonuçlandığını henüz göstermez. Benzer şekilde bir alışverişin başarıyla tamamlanması, normal akışın çalıştığını gösterir; bağlantı kesintisi, tekrar gönderilen ödeme bildirimi veya yoğun trafik hakkında tek başına sonuç vermez.

Bu rehberde **mevcut** kaynak kodda görülen yapıyı, **doğrulandı** belirli bir çalıştırmada gözlenen sonucu, **planlanıyor** henüz uygulanmamış hedefi anlatır. Kaynak incelemesi 22 Ekim 2026 tarihli çalışma kopyasına dayanır. Çalıştırma kanıtları önceki geliştirme oturumlarından, özellikle 21 Ekim'deki temiz ortam denemesinden gelir. Bu belge için yeni bir performans ölçümü yapılmamıştır.

Alan	Mevcut durum ve kanıt sınırı
Mimari	Modüllere ve katmanlara ayrılmış monolith var. Modüller kendi DbContext ve PostgreSQL şemalarına sahip. Bazı modüller arasında Application bağımlılıkları hâlâ bulunuyor.
Kurulum	Projeler net10.0 hedefliyor. SDK seçimi, merkezi paket yönetimi, Dockerfile ve Compose düzenlemeleri yapıldı. Bu aşama sonraki çalışmaların hazır başlangıcıdır.
Temiz ortam	Ayrı PostgreSQL, RabbitMQ, Elasticsearch, API ve Worker kurulumu oluşturuldu. Ayrı ağ ve volume'lar geliştirme kaynaklarından ayrılmayı sağlıyor.
Örnek veri	Temiz veritabanında 3731 ürün bulunduğu doğrulandı. Bu yükleme henüz sabit tohumla üretilecek yük testi veri seti değil.
Arama	Elasticsearch arama endpoint'i çalıştı. Katalog listeleme PostgreSQL üzerinden yapılıyor; bütün katalog akışı Elasticsearch'e taşınmış değil.
Worker	Süresi dolan siparişleri kontrol ediyor. Ürün ve arama olay tüketicileri hâlâ API sürecinde kayıtlı.
Manuel alışveriş	Sandbox ödeme tamamlandı; temiz veritabanında sipariş Paid, ödeme Succeeded olarak görüldü.
Otomatik testler	Son doğrulama kaydında 35 birim testi geçti. Eski oturumlarda 37 sonucu da var; farklı kod anlarının sayıları tek sonuç gibi birleştirilmez. Entegrasyon ve mimari kapsam henüz kurulmuş değil.
Güvenilirlik	Ortak alışveriş transaction'ı, kalıcı outbox, siparişe bağlı rezervasyon ve kalıcı CheckoutAttempt henüz yok.
Performans	Redis cache, k6 senaryoları ve ölçülmüş kapasite raporu henüz yok.
Gözlemleme	Uygulama ve EF Core logları var. Merkezi log, metrik ve trace altyapısı henüz yok.

Sürüm seçiminin doğru okunması

SDK, hedef framework ve NuGet paketleri farklı şeylerdir. SDK projeyi derleyen araç takımıdır. `net10.0`, uygulamanın hedeflediği çalışma platformudur. Paket sürümü ise kullanılan kütüphanenin sürümüdür. Bu nedenle bütün paketlerde “10” yazması gerekmiyor. Mevcut merkezi dosyada SDK 10.0.402, EF Core 9.0.12, Npgsql EF sağlayıcısı 9.0.4, MassTransit 8.5.7, Elastic.Clients.Elasticsearch 9.5.0 ve Iyzipay 2.1.80 kullanılıyor.

Bu sürümlerin derlenebilmesi, hepsinin bakım ve güvenlik süresinin aynı olduğu anlamına gelmez. Yayın öncesinde destek durumu ve güvenlik güncellemeleri yeniden incelenecek; özellikle EF Core/sağlayıcı uyumu kontrollü bir iş olarak ele alınacak. Paketlerin bütününü aynı büyük sürüme zorlamak yerine uyumlu ve desteklenen bir kombinasyon seçilecektir.

Tamamlanan aşamanın sınırı

Kurulum ve konfigürasyon çalışmasını yeniden başlatmıyoruz. Temiz veritabanı oluşturma, migration uygulama, veri yükleme, indeksleme ve alışveriş denemesi sonraki çalışmalar için dayanak oldu. Bununla birlikte farklı bir bilgisayarda uçtan uca kurulum tekrarı, otomatik dağıtım ve geri yükleme tatbikatı ayrı doğrulamalardır. Ayrıca önceki frontend kontrolünde kalan 7 lint hatası vardı; bunların kapandığına ilişkin yeni kanıt olmadan bütün CI kontrolleri yeşil kabul edilmez.

Bu bölümün tamamlanma ölçütü: Her iddianın yanında onun kaynak incelemesi mi, çalıştırma sonucu mu, hedef mi olduğu anlaşılabiliyor. Bugünün başarısı ile yarının kapasite hedefi birbirine karışmıyor.

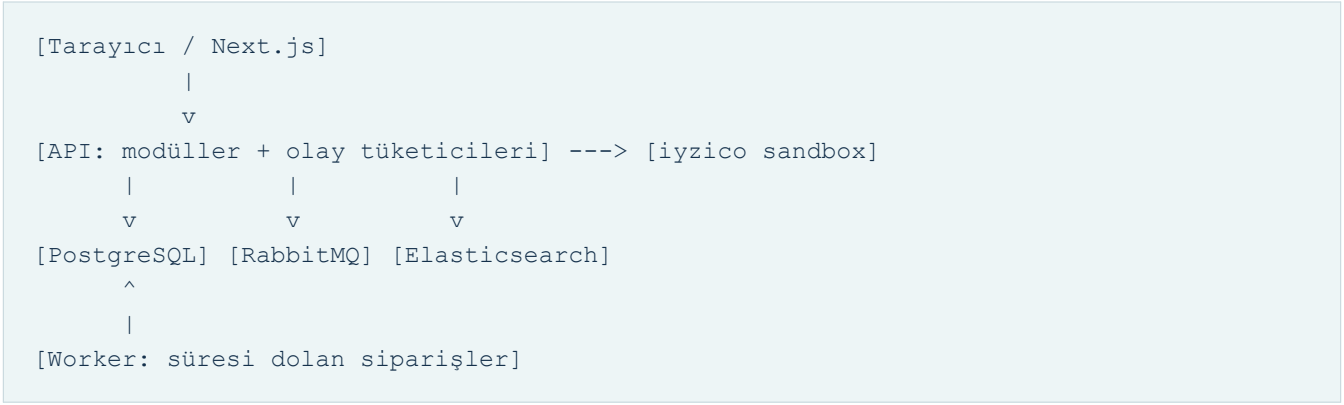
2. Koruyacağımız mimari ve bir alışverişin yolculuğu

Modular monolith ne demek?

Monolith, temel iş uygulamasının tek ürün olarak geliştirilip dağıtılmasıdır. Modular yapı ise kullanıcı, ürün, stok, sipariş ve ödeme gibi sorumlulukların kendi sınırlarına ayrılmasıdır. Ayrı klasörler tek başına modül sınırı oluşturmaz. Başka modülün tablosunu rastgele değiştirmemek, iş kuralını doğru modülde tutmak ve modüller arası iletişimi açık sözleşmelerle yapmak gerekir.

TechCart'ta Domain iş kurallarını, Application kullanım senaryolarını, Infrastructure veritabanı ve dış servis bağlantılarını, Contracts ise dışarıya açılan sözleşmeleri taşıyacak. Mevcut katman ayrımı korunacak. Değiştirdiğimiz akışlarda modüller arası erişim Contracts arayüzleri üzerinden düzenlenecek. Bütün kod tabanının bu kurala zaten uyduğunu varsaymıyoruz.

Mevcut sistemin resmi



Kullanıcı ürün listesini açınca frontend API'yi çağırıyor. Katalog sorgusu PostgreSQL'e gidiyor. Ayrı arama endpoint'i metin aramasında Elasticsearch kullanabiliyor. Sepet, adres, sipariş ve ödeme kayıtlarının doğruluğu

PostgreSQL'e dayanıyor. Stok değişiklikleri mesajlaşma üzerinden diğer görünümlere aktarılıyor. Bugün bu mesajların bazı tüketicileri API içinde çalışıyor.

Hedef sistemin resmi

```
[Tarayıcı] -- HTTPS --> [Caddy]
                         |    |
                         v    v
                    [Next.js] [API] ----> [iyzico sandbox]
                                | \
                                |  +--> [Redis: geçici kopyalar]
                                +-----> [Elasticsearch: arama]
                                v
                          [PostgreSQL]
                          [iş verisi + outbox]
                                |
                                v
                           [Worker] <----> [RabbitMQ]
                              |  |
                              v  v
                          [PG] [Elasticsearch]

[API + Worker + sunucu] --> [Alloy] --> [Grafana Cloud]
```

PostgreSQL, bir siparişin ve ödemenin esas kayıt kaynağıdır. Redis daha hızlı erişim için geçici kopya tutar. Elasticsearch aramaya uygun türetilmiş belge tutar. Redis veya indeks kaybolduğunda PostgreSQL'den yeniden üretilebilirler. Bu yüzden “ürünün gerçekte kaç adedi satılabilir?” sorusunun kesin cevabı cache veya indeks üzerinden verilmez.

Örneğin bir ürünün stok görünümü aramada iki saniye geriden gelebilir. Kullanıcı yine ürünü sepete ekleyebilir; satın alma aşamasında stok modülü koşullu rezervasyon yapar. Son adet başka kişiye gittiyse işlem kontrollü olarak reddedilir. Bu davranış, hızlı bir ekran ile doğru bir satış kararını birlikte sağlar.

Neden mikroservise geçmiyoruz?

Şu anda öncelik işlem doğruluğu, ölçüm ve yayın disiplinidir. Mikroservis; ayrı dağıtımlar, ağ hataları, dağıtık iş akışları ve daha fazla işletim yükü getirir. Tek geliştirici, haftada 8-12 saat ve 30 USD bütçeyle bu maliyetin karşılığını henüz ölçmüş değil. Kafka veya Kubernetes de yalnızca kullanıcı hedefi 1000 olduğu için gerekli olmaz.

Bir modülün bağımsız ölçeklenme ihtiyacı, ayrı ekip sahipliği veya dağıtım engeli ölçülürse ayrıştırma değerlendirilebilir. Önce Worker'ı API'den ayrı çalıştırmak ve tüketici eşzamanlılığını ayarlamak daha küçük bir adımdır. Ayrı süreç kullanmak, bütün sistemi mikroservise dönüştürmek anlamına gelmez.

Doğrulama ve hata noktası: Mimari testler katman bağımlılıklarını izleyecek. “Event kullandım, o hâlde her işlem asenkron olmalı” sonucuna gitmeyeceğiz. Kullanıcıya sipariş sonucunu vermeden önce gereken kesin kararlar senkron ve transaction içinde kalacak.

3. Ortamlar, kurulum ve konfigürasyonun anlamı

Ortam, kodun hangi kaynaklara bağlandığıdır

Aynı sipariş kodu geliştirmede deneme veritabanına, internetteki demoda demo veritabanına, yük testinde ise test veritabanına bağlanabilir. İş mantığını bu geçişlerde değiştirmeyiz. Değişen şey bağlantı adresleri, anahtarlar, kuyruklar, indeksler ve bazı çalışma politikalarıdır. Kullanıcının tarayıcıdan siteye girmesi sunucuda yeni bir kurulum oluşturmaz; kullanıcılar yayımlanan aynı API kopyasını kullanır.

`techcart_clean`, boş bir kurulumun ayağa kalkabildiğini denemek için oluşturulmuş ayrı veritabanıdır. Adı onu otomatik olarak canlı demo ya da kalıcı yük testi ortamı yapmaz. Yük testi için ayrıca ayrılmış veri, anahtar ve kaynaklar gerekecek. Aynı PostgreSQL sunucusunda farklı veritabanı kullanmak veri ayrımı sağlar; aynı makinenin CPU ve diskini paylaşmak ise performans ayrımı sağlamaz.

Ayarlar hangi yoldan geliyor?

Çalıştırma biçimi	Gizli ayarın yolu	Bağlantı adresinin anlamı
Rider, Development	User Secrets içindeki secrets.json	API bilgisayarda çalıştığı için host'a açılan portu kullanır.
Docker Compose	.env veya .env.clean → Compose eşlemesi → container ortam değişkeni	API kendi Compose ağındaki servis adını ve iç portu kullanır.
Gelecek sunucu kurulumu	Sunucuda sağlanan ortam değişkenleri veya gizli değer deposu	Demo kaynaklarına gider; geliştiricinin bilgisayarındaki dosyaları okumaz.

ASP.NET Core'un standart yapılandırmasında ortam değişkenleri JSON ayarlarını, Development User Secrets da daha önce yüklenen JSON değerlerini geçersiz kılabilir. Özel provider eklenirse sıralama yeniden kontrol edilir. `.env` dosyasının varlığı ASP.NET Core'un onu kendiliğinden okuduğu anlamına gelmez. Bizde Compose bu dosyayı okuyup değerleri çalışan sürece verir. Microsoft yapılandırma belgesi

```
# Mevcut yaklaşımın sadeleştirilmiş örneği
environment:
  RabbitMQ__Host: rabbitmq
  Iyzico__ApiKey: "${IYZICO_API_KEY:?IYZICO_API_KEY eksik}"
```

İkinci satır API'ye “RabbitMQ sunucusunun adı rabbitmq” bilgisini verir. Docker'ın ağ içi DNS'i bu adı o ağdaki container IP'sine çözer. IP'yi Compose dosyasına sabitlemeyiz. `localhost`, container içindeyken o container'ın kendisidir. API container'ından PostgreSQL'e giderken bilgisayarın dış portunu kullanmayız.

Temiz kurulumda doğruladığımız bağlantılar

Temiz ortamda bilgisayardan API'ye `localhost:8081`, PostgreSQL'e `localhost:5434`, Elasticsearch'e `localhost:9201`, RabbitMQ yönetimine `localhost:15673` ile erişilir. Container'lar kendi aralarında PostgreSQL için `postgres:5432`, mesajlaşma için `rabbitmq:5672`, arama için `http://elasticsearch:9200` kullanır. Yönetim paneli portu ile mesaj taşıma portu farklıdır.

Volume, container dosya sisteminden bağımsız saklanan veridir. Container silinip yeniden oluşturulduğunda verinin kalmasını sağlar; yedek yerine geçmez. Disk bozulduğunda aynı diskteki volume da kaybolabilir. Ayrı

Compose proje adı, ağ ve volume adlarının ayrılmasına yardımcı olur; açıkça ortak verilen kaynaklar yine paylaşılabilir.

`scripts/migrate-clean.sh`, .NET'in hazır sihirli aracı değildir. Projedeki modüller için `dotnet ef database update` işlemlerini sırayla çağıran bizim shell betiğimizdir. Migration dosyaları tablo değişikliğinin tarifini, bu betik ise tariflerin hangi hedefe hangi sırayla uygulanacağını taşır. Mevcut temiz kurulum betiklerinin `techcart_clean` korumaları vardır; başka ortama olduğu gibi uygulanacaklarını varsaymayız.

Seeder bağlantıyı dış yapılandırmadan alarak aynı kodla seçilen veritabanına veri yükleyebilir. Ancak bugünkü ürün yüklemesi rastgele stok üretebildiğinden deterministik yük testi hazırlığı sayılmaz. Gizli değerlerin Git dışında olması şifrelenmiş olmaları anlamına gelmez. Worker'ın mevcut JSON bağlantı ayarı gibi kalan noktalar yayın öncesi son gizli değer denetiminde ele alınacak; gerçek değerler belgeye taşınmayacaktır.

Doğrulama: Hedef veritabanı adı ve kayıt sayıları kontrol edilir; anahtarın kendisi loglanmaz. Geliştirme, demo ve LoadTest için farklı PostgreSQL, RabbitMQ alanı/kuyrukları ve Elasticsearch indeksleri seçildiği doğrulanır.

4. Siparişin tamamını tek işlem olarak düşünmek

Sorun: her adım ayrı ayrı başarılı olabilir

Bugünkü sipariş oluşturma akışı stok rezervasyonu, sipariş, sipariş kalemleri ve sepet temizliği için farklı kayıt adımları içeriyor. İlk iki adım başarılı, üçüncüsü başarısız olursa veritabanında kalemsiz sipariş veya gereksiz tutulmuş stok kalabilir. Her repository'nin kendi `SaveChanges` çağrısını yapması, alışverişin bütününü otomatik olarak tek işlem yapmaz.

Transaction, belirlediğimiz veritabanı değişikliklerinin birlikte kalıcılaşması ya da birlikte geri alınmasıdır. Hedefimiz kullanıcı adresinin sahipliğini doğruladıktan sonra fiyatları güvenilir kaynaktan okumak, stokları ayırmak, sipariş ve kalemleri yazmak, sepeti temizlemek ve ilgili outbox kayıtlarını aynı transaction içinde tamamlamaktır. Kullanıcıya başarı ancak commit sonrasında bildirilir.

```
BEGIN
  İstek anahtarını kontrol et / sahiplen
  Stokları koşullu olarak rezerve et
  Siparişi ve kalemleri yaz
  Sepeti temizle
  Yayınlanacak olayları outbox'a ekle
  Tekrar istekte dönecek sonucu kaydet
COMMIT

Herhangi bir adım hata verirse -> ROLLBACK
```

Farklı DbContext'ler nasıl aynı işleme katılır?

EF Core, tek bir `SaveChanges` içindeki değişiklikleri destekleyen sağlayıcılarda transaction ile uygular. Birden fazla context arasında ortak işlem için aynı fiziksel `DbConnection` ve aynı `DbTransaction` paylaşılmalıdır. Sadece connection string'lerin aynı olması yeterli değildir; iki context ayrı bağlantı açmış olabilir. EF Core transaction paylaşımı

Hedef bir işlem koordinatörü, bu kullanım senaryosu için bağlantıyı açacak, transaction'ı başlatacak ve katılan context'leri buna bağlayacak. Repository metotları kendi başına yeni transaction başlatmayacak. Domain katmanına Npgsql bağlantısı taşınmayacak; koordinasyon Infrastructure ve kullanım senaryosu sınırında çözülecek. Bu tasarım aynı PostgreSQL veritabanındaki modül şemaları için seçiliyor; farklı sunucular arasında aynı yerel transaction mümkün değildir.

```
// Hedef uygulamanın akış taslağı; projeye eklenmiş kod değildir.
await using var tx = await coordinator.BeginAsync(ct);
await inventory.ReserveForOrderAsync(orderId, lines, ct);
await orders.AddAsync(order, ct);
await orderItems.AddRangeAsync(lines, ct);
await cart.ClearAsync(userId, ct);
await events.StageAsync(orderCreated, ct);
await coordinator.SaveAndCommitAsync(ct);
```

Bu taslakta kritik nokta metot adları değil, bütün veritabanı yazılarının gerçekten aynı bağlantı ve transaction'a katılmasıdır. İnceleme sırasında “koordinatör var” demekle yetinmeyeceğiz. SQL/transaction testinde son adıma hata enjekte edip hiçbir ara kaydın kalmadığını göstereceğiz.

Sınırı doğru seçmek

iyzico çağrısını bu transaction'ın içine koymayız. Dış servis birkaç saniye veya daha uzun beklerse veritabanı kilitleri de bekler. Veritabanı geri alınsa bile dış dünyada yapılmış tahsilatı geri alamayız. Bu nedenle siparişin yerel kaydı kısa bir transaction ile tamamlanır, ödeme başlatma ayrı yönetilir, sonuç yine kısa bir transaction ile işlenir.

PostgreSQL'in varsayılan izolasyonunu bütün sistemi kapsayacak şekilde körlemesine yükseltmek de çözüm değildir. Koşullu güncellemeler, doğru kilit sırası ve belirli durum geçişleri çoğu ihtiyacı karşılayabilir. Transaction retry uygulanırsa bütün kullanım senaryosunun tekrar çalışabileceği hesaba katılmalı; dış ödeme çağrısı bu tekrarın içine sürüklenmemelidir.

Tamamlanma ölçütü: Stok ayrıldıktan, sipariş yazıldıktan ve sepet temizlenmeden hemen önce oluşturulan kontrollü hataların hiçbirinde yarım iş kalmıyor. Başarılı işlemde sipariş kalemleri ve rezervasyonlar toplamlarıyla birlikte tutarlı. Transaction süresi ve kilit beklemeleri ölçülüyor.

5. Stok rezervasyonu ve eşzamanlılık

Stok ile kullanılabilir stok aynı değildir

`stock` eldeki toplam miktarı, `reserved_stock` ödeme bekleyen siparişler için ayrılmış miktarı temsil etsin. Yeni siparişe sunulabilecek miktar `stock - reserved_stock` olur. Örneğin stok 10, rezervasyon 3 ise yeni istekler için kullanılabilir miktar 7'dir. Ödeme başarılı olduğunda toplam stok ve o siparişin rezervasyonu birlikte azalır; süre dolunca yalnızca rezervasyon serbest kalır.

Mevcut koşullu SQL rezervasyonu doğru bir başlangıçtır. “Önce oku, yeterliyse sonra azalt” yaklaşımı ise iki kişinin aynı son adedi görmesine izin verebilir. Kararı atomik bir güncellemeye taşımak gerekir. Aşağıdaki örnek, eklenecek siparişe bağlı rezervasyon modelinin yerine geçmez; eşzamanlılık kararını gösterir.

```
-- Hedef kuralı gösteren sadeleştirilmiş SQL
UPDATE inventory.product_stock
SET reserved_stock = reserved_stock + @quantity
WHERE product_id = @product_id
  AND @quantity > 0
  AND stock - reserved_stock >= @quantity;
-- Etkilenen satır 1: ayrıldı. 0: başarı kabul edilmez.
```

Veritabanında miktarın pozitifliği ve `0 <= reserved_stock <= stock` sınırı da korunur. Yalnızca C# doğrulaması yeterli değildir; başka bir kod yolu veya yarış koşulu aynı tabloyu değiştirebilir. Kısıt ihlali de uygulamada anlamlı bir iş/hata sonucuna çevrilir; kullanıcıya ham SQL mesajı dönülmez.

Rezervasyon hangi siparişe ait?

Bugün yalnızca ürün ve miktarla yapılan onay/serbest bırakma, tekrar gelen istekte başka bir siparişin payını etkileyebilir. Hedef rezervasyon kaydı `OrderId`, `ProductId`, `Quantity`, `ExpiresAt` ve durum taşır. Aynı sipariş ve ürün için tekrar etkisini önleyen bir tekillik kuralı bulunur. Onay sadece Active durumundaki doğru rezervasyona uygulanır; ikinci onay stoktan bir kez daha düşmez.

Olay	Rezervasyonun önceki durumu	Hedef etki
Sipariş oluşturma	Kayıt yok	Active rezervasyon oluştur, ayrılan miktarı artır.
Ödeme başarılı	Active	Confirmed yap, stock ve reserved_stock'u aynı miktarda azalt.
Süre dolması	Active	Released yap, yalnızca reserved_stock'u azalt.
Aynı onayın tekrarı	Confirmed	Önceki sonucu dön; ikinci stok etkisi yaratma.
Geç başarılı ödeme	Released	Tahsilatı kaydet; stok yeniden ayrılamıyorsa uzlaştırmaya al.

Mevcut iptal kodunda süre 15 dakika olarak sabitlenmiş durumda. Hedefte bu değer ayardan yönetilecek. Worker'ın her dakika taraması, her kaydın tam 15. dakikada iptal edileceği anlamına gelmez; tarama aralığı kadar gecikme olabilir. Zamanlar UTC tutulur. Kullanıcının bilgisayar saatine güvenilmez.

Ödeme ile süre sonu aynı anda gelirse

Ödeme callback'i ve Worker aynı sipariş üzerinde yarışabilir. İki yol aynı kilit ve durum geçişi protokolünü kullanmalıdır. Örneğin önce sipariş satırı kilitlenir; sonra rezervasyonlar ürün kimliğine göre tutarlı sırayla alınır. Bir yol AwaitingPayment → Paid yaparken diğeri hâlâ aynı başlangıç durumunda olduğunu varsayamaz. Etkilenen satır sayısı ve rezervasyon durumu kontrol edilir.

Kilit bütün uygulamayı durduran global bir kilit olmamalı. Sadece ilgili sipariş/ürünlerin korunması hedeflenir. Birden fazla ürünlü siparişte her yolun aynı kilit sırasını izlemesi deadlock riskini azaltır. Yine de deadlock veya serialization hatası için sınırlı, güvenli retry gerekir.

Doğrulama: Başlangıç stok 10 iken 100 eşzamanlı, her biri bir adet isteyen deneme yapılır. Aktif rezervasyon ve tamamlanmış satışların toplamı kapasiteyi aşmaz. Test sonunda stok ve rezervasyon negatif olmaz; aynı siparişin ikinci onayı ek düşüm yaratmaz. HTTP 200 saymak bu doğrulamanın yerine geçmez.

6. Ödeme, belirsizlik ve güvenli tekrar

Ödeme başlatmak, ödemeyi almak değildir

Kullanıcı “ödemeye geç” dediğinde sunucu sağlayıcıda bir ödeme oturumu başlatır. Sağlayıcıdan dönen form veya token bir sonraki adım içindir. Kullanıcının kart bilgilerini girmesi, bankanın kabulü ve bizim sonucu kaydetmemiz ayrı aşamalardır. Bu nedenle sipariş oluşturma, checkout başlatma ve ödeme sonucu ayrı izlenmelidir.

Hedef `CheckoutAttempt` kaydı; hangi kullanıcı ve sipariş için, hangi sepet kimliği, tutar, para birimi ve sağlayıcı token'ıyla deneme yapıldığını tutar. Başlatılmadan önce yerel deneme kimliği oluşturulur. Dış çağrı transaction dışında yapılır. Sonuç geldiğinde token ve sağlayıcı referansları kalıcı kayda bağlanır; belirsiz kalan deneme yeniden sorgulanabilir.

Sağlayıcının tarayıcıyı callback'e yönlendirmesi tek başına ödeme kanıtı değildir. iyzico Checkout Form akışında callback token'ıyla sunucudan sonuç yeniden sorgulanır. Bu davranış korunacak; gelen sonuç kalıcı deneme, sipariş/sepet kimliği, tutar ve para birimiyle karşılaştırılacak. URL'deki `orderId` tek başına güven kaynağı olmayacak. iyzico sonuç sorgulama

İdempotency: aynı niyet, tek iş etkisi

Mobil ağ kesilince kullanıcı yeniden basabilir. Tarayıcı veya proxy isteği tekrar gönderebilir. `Idempotency-Key`, aynı iş niyetinin bu tekrarlarında aynı sonucu döndürmek içindir. Sipariş oluşturma ve checkout başlatma ayrı işlemler olduğundan ayrı anahtar alanları kullanılır. Anahtar kullanıcı ve işlem türüyle birlikte saklanır; istek içeriğinin özeti de kaydedilir.

Tekrar durumu	Beklenen davranış
Aynı kullanıcı, işlem, anahtar ve içerik	Önceden kaydedilmiş aynı sipariş/deneme sonucu döner.
Aynı anahtar, farklı içerik	409 Conflict; eski niyet sessizce değiştirilmez.
Aynı anahtarla eşzamanlı iki istek	Veritabanı tekillik kuralı tek sahip belirler; iki sipariş oluşmaz.
İş hâlâ sürüyor	Belgelenmiş “işleniyor” sonucu veya sınırlı bekleme; yeni işlem başlatılmaz.

Frontend, yanıtı kaybettiğinde aynı niyet için aynı anahtarı korumalıdır. Her retry'da yeni UUID üretmek korumayı bozar. Sepet içeriği bilinçli olarak değişirse yeni niyet ve yeni anahtar gerekir. Anahtarların saklama süresi, yanıt biçimi ve temizlik politikası API sözleşmesinde belirlenir. Idempotency, kullanıcı yetkilendirmesinin yerine geçmez.

Zaman aşımı ile başarısızlık farklıdır

Sağlayıcı isteği zaman aşımına uğradığında “para alınmadı” sonucunu kesin bilmiyor olabiliriz. Hemen ikinci tahsilata yönelmek yerine denemeyi Unknown/Pending gibi bir durumda saklar, sağlayıcı referansıyla sorgularız. Sağlayıcının kendi tekrar güvenliği kuralları ayrıca uygulanır. Yerel idempotency anahtarı, dış sağlayıcının otomatik olarak aynı semantiği sunduğu anlamına gelmez.

Ödeme başarıyla alınmış fakat sipariş süresi dolmuş olabilir. Bu durumda finansal gerçek “tahsilat başarılı”dır. Siparişin iptal edilmiş olması bu kaydı Failed yapmamıza neden olmamalıdır. Hedef tasarım ödeme başarısını korur; siparişi uzlaştırma gereken bir duruma alır. Stok yeniden ayrılabiliyorsa kontrollü tamamlama, ayrılamıyorsa iade/operasyon kararı gerekir. Gerçek satış kapısında bu prosedür tamamlanmış olmalıdır.

Tutarlar para türünde/decimal olarak ve tanımlı yuvarlama politikasıyla karşılaştırılır. Eksik veya geçersiz sağlayıcı alanı güvenli biçimde ele alınır. Callback token'ı, kart bilgileri ve anahtarlar loglanmaz. Sonuç sayfası `status=success` parametresine inanmak yerine yetkili kullanıcının sipariş durumunu API'den okur.

Tamamlanma ölçütü: Aynı callback eşzamanlı gönderildiğinde tek ödeme etkisi oluşur. Başlatma yanıtı kaybolduğunda ikinci sipariş veya tahsilat denemesi kontrolsüz açılmaz. Süre sonu ile başarı yarışında tahsilat kaydı kaybolmaz ve stok kuralı bozulmaz.

7. Kaybolmayan olaylar: outbox ve inbox

Publish çağrısı neden yeterli değil?

Siparişi veritabanına yazıp sonra RabbitMQ'ya olay yayımladığımızı düşünelim. Commit başarılı olduktan hemen sonra API kapanırsa sipariş vardır ama olay yoktur. Sırayı ters çevirirsek bu kez mesaj ulaşabilir fakat veritabanı işlemi geri alınabilir. İki ayrı sistemi arka arkaya çağırmak tek atomik işlem oluşturmaz.

Outbox, yayımlanacak olayın önce iş verisiyle birlikte aynı veritabanı transaction'ında saklanmasıdır. Commit olduğunda hem sipariş hem gönderilecek olay kalır. Worker daha sonra bu kalıcı kaydı RabbitMQ'ya teslim eder. RabbitMQ kapalıysa iş kaydı kaybolmaz; bekler ve bağlantı geldiğinde yeniden denenir. Kullanıcıya “olay bütün tüketicilerde işlendi” sözü verilmez; kalıcı teslim sorumluluğu alınmış olur.

```
API transaction'ı:
  [Sipariş + rezervasyon + outbox mesajı] -> COMMIT

Worker teslimatı:
  [Bekleyen outbox] -> RabbitMQ -> [Tüketici]
           ^                |
           +-- retry -----+

Tüketici transaction'ı:
  [Inbox kontrolü + yerel iş etkisi] -> COMMIT -> ACK
```

TechCart'ta seçilen yol

Mevcut MassTransit 8 serisi korunacak; EF tabanlı kalıcı outbox kullanılacak. `MassTransit.EntityFrameworkCore` 8.5.7 paketi mevcut seriyle hizalanabilir. Ancak bir paketi eklemek tek başına atomiklik sağlamaz. Ortak messaging context'i, iş context'leriyle aynı bağlantı ve transaction'a katılmalıdır. Kendi başına ayrı `SaveChanges` yapan bir messaging context aynı sorunu yeniden üretir. MassTransit outbox, 8.5.7 paket kaydı

API içindeki ürün ve Elasticsearch tüketicileri Worker'a taşınacak. API'nin görevi isteği ve yerel iş kurallarını tamamlamak; Worker'ın görevi teslimat, tüketim ve bakım işlerini yürütmek olacak. Outbox kayıtlarının kime ait olduğu, kaç worker'ın aynı kaydı alabileceği ve gönderim sırası kütüphanenin kalıcı outbox mekanizması üzerinden yönetilecek; elle ikinci bir kuyruk algoritması yazılmayacak.

Aynı mesaj niçin yeniden gelebilir?

Worker mesajı RabbitMQ'ya iletti, fakat “gönderildi” kaydını güncellemeden kapandı. Yeniden açıldığında mesaj tekrar gönderilebilir. Tüketici veritabanına yazdı, fakat ACK vermeden kapandıysa broker tekrar teslim edebilir. Bu yüzden outbox, “her mesaj sadece bir kere görülecek” garantisi değildir. Hedef en az bir kez teslim ve tekrarın güvenli işlenmesidir.

Inbox, tüketicinin hangi mesajı işlediğini izler. Aynı tüketici ve MessageId için tekillik kuralı bulunur. Veritabanı projeksiyonundaki değişiklik ile inbox işaretinin aynı transaction'da olması gerekir. Önce “işlendi” deyip sonra veriyi yazmak, aradaki çökmede mesajı kaybettirir. Önce veriyi yazıp ayrı transaction'da işaretlemek ise çift etki üretebilir.

Elasticsearch, PostgreSQL transaction'ına katılmaz. Burada aynı EventId'yi izlemek tek başına bütün atomiklik sorununu çözmez; belge yazısının tekrar uygulanabilir ve sürüm korumalı olması gerekir. “Stoku bir azalt” yerine belirli sürüm için hesaplanmış yeni görünümü yazmak tekrar teslim açısından daha güvenlidir.

Doğrulama: Broker kapalıyken sipariş ve outbox commit edilir. Süreç kapatılıp açılır. Broker döndüğünde olay teslim edilir ve outbox yaşı normale iner. Aynı mesaj birden çok kez teslim edilse de tek iş etkisi görülür. Henüz kalıcı outbox yokken bu dayanıklılık kanıtlanmış sayılmaz.

8. Tekrar, sıra ve hata kuyruğu yönetimi

Dört kimliğin dört ayrı görevi var

Alan	Anlamı	Kullanımı
EventId	Belirli olayın benzersiz kimliği	Tekrar işleme kontrolü, olay kaydı ve hata inceleme.
CorrelationId	Bir iş akışını ilişkilendiren kimlik	Aynı checkout/sipariş çevresindeki mesajları bağlama.
TraceId	Teknik iz zincirinin kimliği	İstek ve ona bağlı span'lerde zamanın nerede geçtiğini bulma.
StockVersion	Ürünün stok değişim sırası	Eski görünümün yeni görünümü geri almamasını sağlama.

TraceId'yi sipariş numarası yerine kullanmayız. Bir sipariş için farklı zamanlarda farklı HTTP istekleri ve trace'ler oluşabilir. Outbox gecikmeyle gönderdiği için iz bağlamı mesaj başlıklarında korunur; uzun gecikmede tüketim izini önceki işlemle link'lemek gerekebilir. CorrelationId ve iş kimliği bu zamanlar arasında sürekliliği sağlar.

Sırası değişen stok olayları

Bir ürünün sürüm 41'de kullanılabilir stoku 8, sürüm 42'de 6 olsun. İndeks önce 42'yi, sonra 41'i alırsa stok tekrar 8 görünmemelidir. Bu nedenle stok sürümü PostgreSQL'de stok değişikliğiyle aynı işlemde artar. Tüketici yalnızca daha yeni sürümü uygular; aynı sürümün tekrarı etkisizdir. Elasticsearch'te dış sürümleme veya karşılaştırmalı güncelleme bu kuralı uygulayabilir; eşit sürüm politikası da test edilir.

Olayın zamanı sıralama yerine kullanılmaz. Saatler kayabilir ve aynı milisaniyede birden çok değişim olabilir. Ürün bazında monoton artan sürüm daha açık bir sıradır. Rezervasyon, iptal ve satış olaylarının hepsi aynı `stock - reserved_stock` tanımından türetilmelidir; aksi hâlde ayrı projeksiyonlar farklı stok gösterebilir.

Hataları yutmak neyi değiştirir?

Mevcut Elasticsearch tüketicisi hatayı loglayıp normal dönebiliyor. Mesajlaşma sistemi açısından tüketici bitmiş göründüğü için mesaj başarılı sayılabilir. Hedefte geçici bağımlılık hatası tekrar denenebilecek biçimde dışarı taşınacak. Doğrulama/şema hatası gibi kalıcı bir sorunu ise sonsuz hızlı retry ile döndürmeyeceğiz.

Başlangıç retry politikası, örneğin kısa bir deneme ve ardından artan aralıklarla sınırlı redelivery olabilir. Kesin sayılar hata deneyiyle seçilir. Sürekli erişilemeyen bir bağımlılık için bellekte bekleyen retry'lar tüketici slotlarını doldurmamalı. Tüketici eşzamanlılığı veritabanı havuzunu aşmayacak şekilde sınırlanır. Amaç en fazla iş parçacığını açmak değil, kuyruk büyümeden sürdürülebilen hızı bulmaktır.

Hata kuyruğu için uygulanacak prosedür

1. Hata sayısı ve en eski mesaj yaşı panelden görülür; etkilenen olay türü belirlenir.
2. EventId, sürüm, hata sınıfı ve kod sürümü incelenir. Gizli veri içeren payload panellere kopyalanmaz.
3. Bağımlılık veya kod düzeltilir; test ortamında aynı mesaj tekrar işlenir.
4. Sınırlı bir grup yeniden gönderilir. Orijinal mesaj kimliği korunur; idempotency kontrolü aşılmaz.

5. Projeksiyon ile esas veri karşılaştırılır, kuyruk ve outbox birikiminin azaldığı doğrulanır.

Kuyruğu silmek hatayı çözmez, kanıtı ve işi yok eder. Bütün mesajları sınırsız hızla yeniden basmak da ikinci bir kesinti yaratabilir. Yeniden işleme kayıtlı ve kontrollü bir operasyon olacaktır.

MassTransit'in 9 ve sonraki sürümleri için ticari lisans gereksinimi açıklanmıştır. Bu nedenle mevcut 8 serisini sırf büyük numara arttı diye yükseltmiyoruz. Güvenlik, bakım ve lisans koşulları yayın öncesinde ayrıca değerlendirilir; 8 serisinin sonsuza kadar yeterli olduğu da varsayılmaz. MassTransit lisans açıklaması

Tamamlanma ölçütü: Eski olay yeni olaydan sonra verildiğinde görünüm geriye gitmiyor. Geçici hata toparlanıyor; kalıcı hata görünür bir hata kuyruğuna gidiyor. Yeniden işleme çift etki yaratmıyor ve operatör hangi mesajı neden yeniden işlediğini açıklayabiliyor.

9. Log, metrik ve trace ile sistemi anlamak

Aynı sorun için üç farklı bakış

Bir kullanıcı “ödemeye bastım, çok bekledi” dediğinde yalnızca hata mesajı yeterli olmaz. Kaç kişinin etkilendiğini, gecikmenin hangi adımda oluştuğunu ve bu siparişe ne olduğunu bilmek isteriz. Log, metrik ve trace bu soruların farklı parçalarını cevaplar.

Sinyal	Sorduğumuz soru	TechCart örneği
Log	Ne oldu, hangi bağlamda oldu?	Checkout sonucu belirsiz kaldı; sağlayıcı çağrısı zaman aşımına uğradı.
Metrik	Ne sıklıkta oluyor, ne kadar yavaş?	Son beş dakikada checkout teknik hata oranı ve p95 süresi arttı.
Trace	Bu istek hangi adımda zaman harcadı?	180 ms veritabanı, 3 saniye sağlayıcı, 20 ms yerel sonuç kaydı.

Log tekil olay kaydıdır. Metrik sayısal zaman serisidir; binlerce isteği özetler. Trace, bir işlemin içindeki span'leri ilişkilendirir. Span, örneğin HTTP isteği, SQL sorgusu veya ödeme çağrısı gibi zaman aralığı olan bir adımdır. “API yavaş” ifadesini “sağlayıcı çağrısı bekledi, veritabanı normaldi” diye somutlaştırır.

Hedef uygulama kapsamı

Mevcut `ILogger` kullanımı yapılandırılmış JSON çıktısıyla devam edecek. OpenTelemetry SDK; ASP.NET Core istekleri, HttpClient dış çağrıları, Npgsql veritabanı işlemleri ve mesajlaşma izlerini toplayacak. Kritik iş adımlarına, örneğin stok rezervasyonu ve ödeme sonucu işlemesine, gerekli olduğunda özel span eklenecek. Her metodu ayrı span yapmak gürültü ve maliyet üretir. OpenTelemetry .NET

```
{
  "level": "Warning",
  "event": "CheckoutOutcomeUnknown",
  "service": "techcart-api",
  "environment": "LoadTest",
  "trace_id": "ornek-teknik-iz",
  "correlation_id": "ornek-is-akisi",
  "provider": "test-gateway",
  "duration_ms": 3000
}
```

Bu örnek gerçek log kaydı değildir. Mesajı cümle birleştirerek üretmek yerine alanlı yazmak filtrelemeyi kolaylaştırır. Aktif iz bağlamındaki TraceId ve SpanId loga iliştirilebilir. Log ekranından trace'e, trace'ten yavaş SQL adımına geçmek hedeflenir. Log ve trace ilişkilendirme

İş kaydı ile telemetri aynı şey değil

Ödeme başarısı veritabanında kalıcı iş kaydıdır. Telemetri saklama süresi doldu diye ödeme gerçeği kaybolamaz. Log sistemi bir finansal defter yerine kullanılmaz. Benzer biçimde logdaki “ödeme başarılı” cümlesi tek başına transaction'ın commit edildiğini kanıtlamaz; logun hangi aşamada yazıldığı önemlidir.

Parola, JWT, cookie, ödeme token'ı, API anahtarı, kart bilgileri ve açık adres log veya span içine taşınmayacak. HTTP gövdelerini toptan kaydetmek kolay görünür ama bu bilgileri sızdırabilir. SQL hassas veri loglaması yayın ortamında kapalı kalır. URL query parametreleri ve exception mesajlarının da gizli veri taşıyabileceği unutulmaz. Kontrol sadece normal log mesajıyla sınırlı tutulmaz.

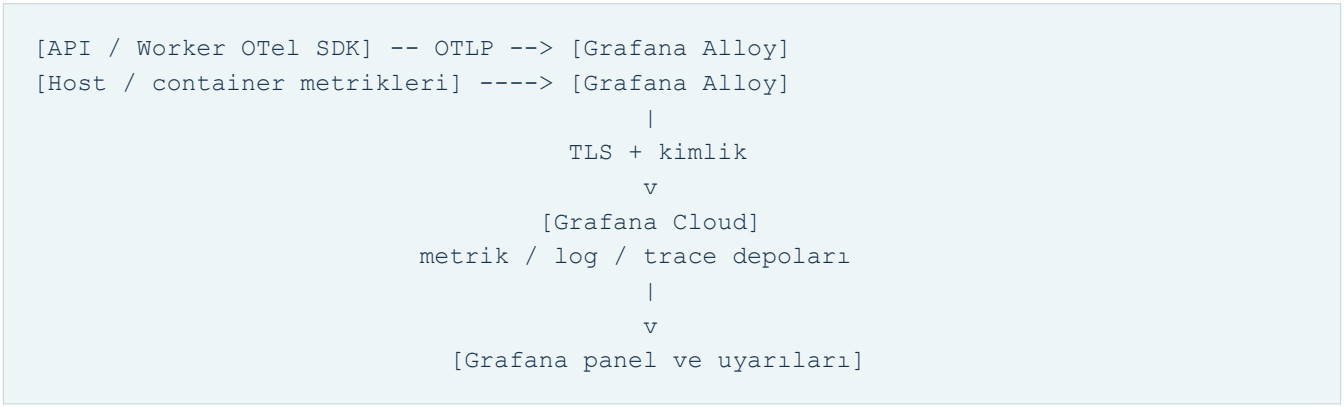
Nasıl doğrulayacağız?

Kontrollü bir ödeme adaptörü gecikmesi oluştururuz. Panelde ilgili endpoint gecikmesi yükselmeli, trace'te yavaş adım dış çağrı olmalı, log kaydında aynı teknik iz bulunmalıdır. Aynı deneyde token ve adresin hiçbir sinyalde görünmediği kontrol edilir. Bu hata üretimi test ortamında yapılır; bu belge hazırlanırken çalıştırılmamıştır.

Düşünme kuralı: Yeni bir log veya metriği eklerken “Bunu gördüğümde hangi kararı vereceğim?” diye sor. Cevabı olmayan ayrıntı, maliyetli gürültü olabilir. Başarılı her sorguyu kalıcı loga yazmak yerine toplam süre ve hata sinyallerini kullanmak çoğu durumda daha uygundur.

10. Gözlemleme altyapısı, paneller ve maliyet

Seçilen akış



OpenTelemetry veriyi üretme ve taşıma standardıdır; tek başına geçmişi tutan bir panel sistemi değildir. Alloy sunucuda telemetriyi toplar, gruplar, filtreler ve iletir. Grafana Cloud'da Prometheus uyumlu metrik hizmeti, Loki logları ve Tempo izleri depolar; Grafana bunları görselleştirir. Böylece VPS'te bütün gözlemleme depolarını ayrıca işletmeyiz. Bu karar RAM, disk ve bakım bütçesini uygulamaya ayırmayı sağlar. Grafana'nın önerilen OTLP mimarisi

Alloy yine kaynak kullanır. Batch, bellek sınırı ve gönderim kuyruğu ayarlanır. Bulut erişilemezse telemetri sınırlı tamponla işlenir; checkout isteği sonsuza kadar exporter beklemez. Sınırlar dolduğunda telemetri kaybı görünür olmalı; alışverişin başarısı bulut log deposunun o anda çalışmasına bağımlı olmamalıdır.

İlk panelde hangi değerler olacak?

Alan	Ölçüm	Ne anlatır?
HTTP	RPS, p50/p95/p99, teknik hata ve timeout	Trafik artınca kullanıcı deneyiminin bozulduğu nokta.
Uygulama	CPU, RAM, GC duraklaması, thread pool	Kaynak doygunluğu veya birikme ihtimali.
PostgreSQL	Havuz kullanımı/bekleme, sorgu süresi, kilit bekleme	Yavaşlığın sorgu mu bağlantı mı kilit mi olduğu.
Mesajlaşma	Ready/unacked mesaj, hata sayısı, en eski iş yaşı	Tüketicinin trafiğe yetişip yetişmediği.
Outbox	Bekleyen sayı ve en eski teslim edilmemiş kaydın yaşı	Broker'a ulaşmamış işlerin etkisi.
Projeksiyon	Olay oluşumu ile görünüm güncellemesi arasındaki süre	Arama/stok görünümünün güncelliği.
Redis	Hit, miss, hata, fallback ve doldurma süresi	Cache'in gerçekten iş yükünü azaltıp azaltmadığı.

Projeksiyon gecikmesi `işlendiği an - OccurredAt` şeklinde kaydedilebilir; saatler senkronize olmalı. Metrik histogram'ı p95 üretir. Hiç olay tüketilmediğinde yeni histogram örneği gelmeyeceğinden ayrıca en eski bekleyen olay/outbox yaşı izlenir. Sessizlik her zaman sağlık demek değildir.

Kardinalite ve örnekleme

Bir metriğin etiket kombinasyonları arttıkça zaman serisi sayısı büyür. `route=/api/orders/{id}`, method, status sınıfı ve environment makul etiketlerdir. Her `orderId`, `userId` veya tam URL'yi metrik etiketi yapmak binlerce seri doğurur. İş kimliği gerekiyorsa erişimi sınırlı log/span alanında, kişisel veri politikasına uygun biçimde kullanılır. Loki etiketlerinde de sınırsız kimliklerden kaçınılır.

Trace örneklemesi hacmi azaltır. Küçük smoke testinde bütün izler, yoğun testte örneğin bir kısmı saklanabilir. Ancak başlangıçta %10 seçen head sampling, sonradan hata veren bütün istekleri saklamayı garanti etmez. Bütün hataları seçmek için yeterli tamponu olan tail sampling gibi bir yaklaşım gerekir. “%10 iz + bütün hatalar” ifadesi yapılandırma olmadan verilmez. Metrik sayaçları ise trace örneklemesine göre eksiltilmemelidir.

Ücretsiz katman sınırsız değil

22 Ekim 2026'da kontrol edilen Grafana fiyat sayfasında ücretsiz katman; 10.000 aktif metrik serisi, aylık 50 GB log alımı, aylık 50 GB trace alımı ve 14 gün saklama ile listeleniyor. Kotalar hizmete özgüdür ve değişebilir. Hesap açılırken güncel koşullar yeniden kontrol edilir. Pro planın 19 USD/ay taban ücreti ve kullanım bedeli 30 USD toplam bütçenin önemli kısmını tüketebilir. Grafana Cloud fiyatlandırması

Yerel k6 açık kaynak çalıştırması ile Grafana Cloud'un ücretli/limitli yük üretme hizmeti ayrı şeylerdir. 1000 VU hedefi için buluttaki ücretsiz test kotasının yeterli olduğu varsayılmayacak. Önce ayrı bir yük üreticisinden yerel k6 çalıştıracağız; buluta gönderilecek sonuçların da hacmini sınırlayacağız.

Tamamlanma ölçütü: Panel bir isteği, bir dış bağımlılığı ve bir mesajı ilişkilendiriyor. Kota kullanımı görünür, günlük log hacmi izleniyor. Uyarının yanında uygulanacak adım var: örneğin outbox yaşı yükselirse broker erişimi, teslimat worker'ı ve hata kuyruğu sırayla inceleniyor.

11. PostgreSQL performansını doğru ölçmek

Her okuma bir problem değildir

İndeksli bir sorgu küçük bir veri kümesini milisaniyeler içinde döndürebilir. Cache eklemek ise ağ çağrısı, serialize/deserialize, bellek ve güncellik yönetimi getirir. Dolayısıyla “veritabanına gittik, kötü oldu” yerine “bu sorgu ne sıklıkta çalışıyor ve toplam ne maliyet üretiyor?” diye sorarız. Mevcut katalog loglarında sayfa için ürünler ve ilişkili bilgiler toplu sorgularla çekilebiliyor; bunu her satır için ayrı sorgu varmış gibi anlatmayız.

İlk ölçümde endpoint başına sorgu sayısı, toplam SQL süresi, dönen satır sayısı, veri büyüklüğü ve bağlantı bekleme süresi kaydedilir. Yavaş bir endpoint'in süresinin çoğu iyzico'da geçiyorsa ürün tablosuna indeks eklemek çözmez. p95 artışının SQL trace'iyle birlikte incelenmesi bu ayrımı sağlar.

Sorgu istatistikleri ve plan

`pg_stat_statements`, benzer sorguları birleştirerek çağrı ve süre istatistiği sunar. PostgreSQL 17'de modülün `shared_preload_libraries` ile yüklenmesi sunucu yeniden başlatması gerektirebilir; uzantı hedef veritabanında da oluşturulur. Bu bir sonraki uygulama işidir, mevcut temiz kurulumda etkin olduğu varsayılmıyor. PostgreSQL 17 sorgu istatistikleri

```
-- Hedef test ortamında, önce gerçek sorgu ve sütunlarla uyarlanır.
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, created_at, total, status
FROM orders.orders
WHERE user_id = '00000000-0000-0000-0000-000000000001'
ORDER BY created_at DESC
LIMIT 20;
```

`ANALYZE` sorguyu gerçekten çalıştırır. Bu nedenle örnek okuma sorgusuyla gösteriliyor; üretimde UPDATE/DELETE üzerinde düşünmeden kullanılmaz. Gerçek plan satır tahminleri, taranan satırlar, sıralama maliyeti ve buffer erişimleri birlikte değerlendirilir. Küçük tabloda sequential scan bazen doğru seçimdir; plan içinde “index” yazmaması tek başına hata değildir. EXPLAIN kullanımı

İndeks ve sayfalama kararları

Sipariş geçmişi için `(user_id, created_at)`, süresi dolan sipariş taraması için durum ve zaman alanları ilk adaylardır; kesin indeks gerçek sorgu ve planla seçilir. Çok indeks yazma maliyetini ve disk kullanımını artırır. Mevcut indeksle aynı olan yeni indeks eklemekten kaçınırız. Büyük canlı tabloda indeks oluşturmanın kilit etkisi ve uygun olduğunda concurrent oluşturma yöntemi ayrıca planlanır.

Salt okuma yollarında `AsNoTracking`, yalnız gerekli alanların projeksiyonu ve sayfa boyutu sınırı kullanılabilir. Çok derin OFFSET sayfaları pahalılaşırsa keyset/cursor sayfalama değerlendirilir; ilk sayfayı göstermek için API sözleşmesi gereksiz yere değiştirilmez. Sıralama kararlı olmalıdır: aynı zamana sahip satırlarda kimlik gibi ikinci sıralama alanı sayfa atlamalarını önler.

Bağlantı havuzu ve kilit bekleme

API, Worker ve yönetim araçları aynı PostgreSQL kaynaklarını paylaşır. Her süreç için yüksek bir maksimum havuz verince toplam bağlantı sınırı aşılabilir. Başlangıç sınırları birlikte hesaplanır; testte bekleme ve aktif bağlantı sayıları izlenir. Havuz dolduğunda sınırsız yeni iş kabul etmek yerine zaman aşımı ve kontrollü geri basınç gerekir. Daha fazla bağlantı her zaman daha çok throughput sağlamaz. Npgsql havuz ayarları

Ölçüm ve tamamlanma: Aynı veri ve yükle sorgu değişikliğinden önce/sonra p95, toplam SQL süresi, çağrı sayısı, CPU ve kilit beklemeleri karşılaştırılır. Sadece tek kişinin bilgisayarında bir kez daha hızlı görünmesi

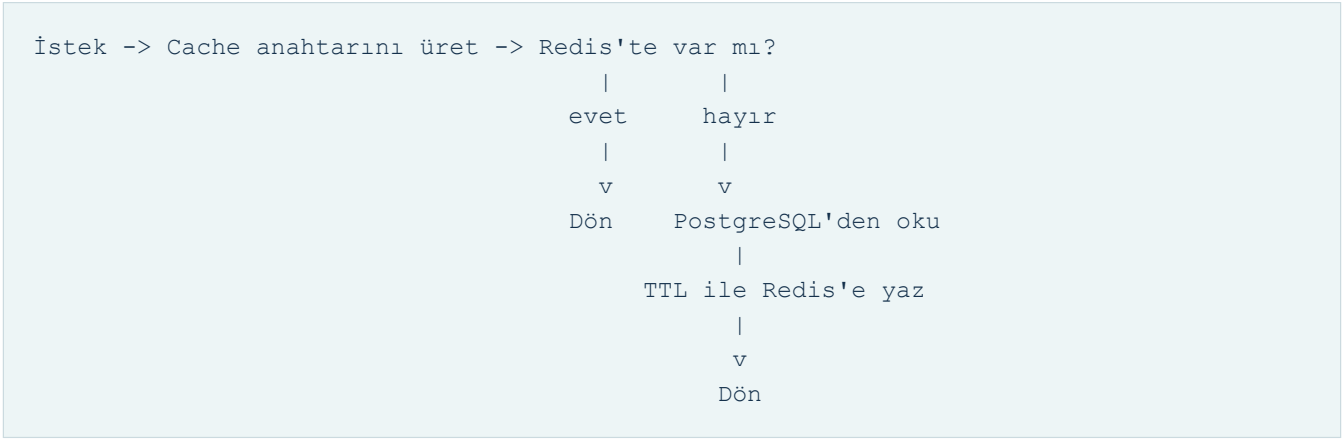
kazanım kanıtı sayılmaz. Düşen okuma maliyetinin yazma maliyetini kabul edilemez artırmadığı da kontrol edilir.

12. Redis cache: nerede, neden ve nasıl?

Çözdüğü sorun tekrar eden aynı okumadır

Bin kişi aynı marka listesini açtığında her istekte aynı az değişen bilgiyi tekrar sorgulamak gerekmeyebilir. Redis, bu sonucu kısa süre saklayarak PostgreSQL yükünü azaltabilir. Ancak sipariş durumunun eski kalması veya ödeme kararının cache'ten verilmesi farklı bir risktir. Cache'i önce yeniden üretilebilir katalog verisine uygulayacağız.

Seçilen başlangıç yaklaşımı **cache-aside**: önce cache'e bak, yoksa güvenilir kaynaktan oku, sonucu süreli kaydet. Uygulama `IDistributedCache` arayüzünü kullanacak; Redis erişimi altyapı katmanında kalacak. Buradaki cache uygulama verisi içindir; HTTP response/output cache ile aynı sorumluluk değildir. ASP.NET Core dağıtık cache



Veri	İlk TTL	Güncellik yaklaşımı
Kategori ve marka listeleri	1 saat	Yönetim değişikliğinde geçersizleştir.
Ürün açıklaması, görseli, katalog bilgisi	5 dakika	Ürün değişikliği sonrası ilgili kaydı kaldır/versiyonla.
Tekrarlanan liste ve facet sonuçları	30 saniye	Bütün filtreleri anahtara kat; kısa eski veri toleransı.
Sipariş, ödeme, kesin fiyat/stok kararı	Cache kullanılmaz	Yetkili ve güvenilir kaynaktan oku.

Anahtar ve geçersizleştirme

`products:list:v1:category=...:brand=...:sort=...:page=...:size=...` gibi bir anahtar düşün. Arama metni, renk, fiyat aralığı ve diğer bütün sonucu değiştiren alanlar da kapsanır. Alan sırası ve normalizasyon sabit olmalı; çok uzun filtreler kanonik içeriğin hash'iyle kısaltılabilir. Kullanıcıya özel sonuçları genel anahtara koymak başka kullanıcının verisini gösterebilir. Bu rehberin ilk cache kapsamı kişisel veri içermeyen katalogdur.

TTL, kaydın en fazla ne kadar tutulacağını belirtir. Değişiklik sonrası invalidation ise ilgili kopyayı daha erken geçersiz kılar. Yalnız “değişince sileriz” demek yeterli değildir: bir eşzamanlı okuma eski veriyi silmeden sonra yeniden yazabilir. Kısa TTL, katalog sürümü veya anahtar nesliyle yapılan versiyonlama bu yarışa göre seçilir.

Liste anahtarlarının hepsini tek tek bulup silmek yerine katalog nesli değiştirmek değerlendirilebilir; eski nesil TTL ile temizlenir.

Aynı anda dolan anahtarlar ve Redis kesintisi

Bir sıcak ürünün TTL'i dolunca 500 istek aynı anda PostgreSQL'e giderse cache stampede oluşur. TTL'e küçük rastgele fark eklemek farklı anahtarların aynı anda düşmesini azaltır. Aynı anahtarın doldurulması uygulama içinde single-flight ile birleştirilebilir: bir istek veriyi getirirken diğerleri kısa süre aynı sonucu bekler. Birden fazla API kopyasında bu yalnız o süreç için geçerlidir; dağıtık kilit ihtiyacı ölçülmeden eklenmez.

Redis kapanınca kısa zaman aşımıyla PostgreSQL'e dönmek kullanıcı akışını koruyabilir. Fakat bütün trafiği bir anda sınırsız sorguya çevirmek veritabanını da düşürebilir. Fallback eşzamanlılığı sınırlanır; başarısız cache çağrıları için kontrollü devre kesme uygulanabilir. Kullanılabilecek eski katalog kopyasının süresi açıkça tanımlanır. Checkout kararı bu esnekliğe dahil değildir.

Cache'in işe yaradığını nasıl göstereceğiz?

Aynı commit, veri seti ve sunucu kaynaklarıyla cache kapalı, cache soğuk ve cache sıcak testleri yapılır. p95/p99, RPS, PostgreSQL çağrıları, Redis bellek kullanımı, hit/miss ve fallback ölçülür. Çok yüksek hit oranı tek başına başarı değildir; yanlış anahtar da herkese aynı sonucu verip hit oranını yükseltebilir. Filtre ve kullanıcı ayrımı ayrıca test edilir.

Tamamlanma ölçütü: Ölçülen fayda maliyetinden büyük, katalog değişikliği belirlenen güncellik süresinde görülüyor, Redis kesildiğinde sistem kontrollü davranıyor. “Redis ekledik, artık 1000 kullanıcı taşır” sonucu çıkarılmıyor; kapasite yeniden ölçülüyor.

13. Elasticsearch ve frontend istek davranışı

Elasticsearch neden cache değil?

Redis'te çoğunlukla daha önce hesaplanmış bir cevabı anahtarla ararız. Elasticsearch ise metni analiz ederek arama, uygunluk sıralaması ve filtreleme yapar. Aynı ürün verisinin farklı bir sorgulama modelidir. PostgreSQL esas kayıt kaynağı, Elasticsearch bu kayıtlardan üretilen arama projeksiyonudur. İndeksin yeniden kurulabilmesi bu ayrımın sonucudur.

Bugün `/api/products` katalog listesinin PostgreSQL'den gelmesi normaldir. Ayrı `/api/search/products?q=samsung&page=1&pageSize=1` çağrısı Elasticsearch üzerinden çalıştı; önceki temiz ortam doğrulamasında 3731 indeks belgesi ve “samsung” için 134 toplam sonuç görüldü. Bunlar mevcut veri anının sonuçlarıdır; evrensel doğrulama sabiti değildir.

Hedef sorgu davranışı

Metin araması olduğunda Elasticsearch adaptörü kullanılacak; filtre, sıralama ve facet'ler aynı sorgu kaynağı üzerinden hesaplanacak. Sonuçlar ES'den, kategori sayıları başka kurallarla PostgreSQL'den gelirse kullanıcı tutarsız bir filtre ekranı görebilir. İndekse kategori/marka kimlikleri, gerekli filtre alanları ve stok sürümü eklenecek. Mevcut ürün listeleme/facet sözleşmelerinin gereksiz yere kırılmaması hedeflenir.

Elasticsearch erişilemezse arama için kontrollü geçici hata gösterilebilir; normal katalog gezintisi PostgreSQL üzerinden devam edebilir. Bütün uygulamayı “hazır değil” yapıp sipariş akışını kapatmak gerekmiyor. Fallback olarak PostgreSQL metin araması seçilirse bunun farklı sıralama ve kapasite davranışı açıkça belgelenir; sessizce eşdeğer sayılmaz.

Canlı kullanımda yeniden indeksleme

Mevcut yeniden indeksleme akışı indeksi silip yeniden oluşturabiliyor. Bu işlem sırasında arama boş veya hatalı dönebilir. Hedefte `products-20261022-01` gibi yeni fiziksel indeks hazırlanır; okuyucu alias üzerinden erişir. Veri yüklenirken oluşan değişiklikler watermark/outbox kaydı ve ürün sürümüyle yakalanır. Güncel duruma yetişildiğinde alias tek atomik işlemle yeni indekse çevrilir. Eski indeks kısa geri dönüş penceresinde tutulabilir. Elasticsearch alias yönetimi

Önemli geçiş ayrıntısı: Bugünkü `products` somut indeks adıyla aynı isimde alias eşzamanlı var olamaz. İlk alias geçişi ayrıca planlanır; sadece “alias ekledik” demek yeterli değildir. İndeks yüklerken gelen olayları kaçırmak da alias'ın atomik olmasıyla çözülmez. Sürüm kontrolü ve değişiklik yakalama bu nedenle birlikte tasarlanır.

Frontend'in trafik üzerindeki etkisi

Arama kutusunda her tuşta istek atmak yerine 300 ms debounce kullanılır. Kullanıcı yeni karakter yazdığında eski HTTP isteği AbortController ile iptal edilir; geç dönen eski yanıtın yeni sonucu ezmesi de önlenir. Debounce gereksiz istekleri azaltır, tek başına sunucunun kapasite problemini çözmez.

Next.js'in önbellekleme davranışıyla Redis'i birbirine karıştırmayız. Sunucu render'ı, browser cache'i ve uygulama cache'i farklı katmanlardır. Bu projedeki Next.js sürümü için kod yazmadan önce yerel rehberler okunacak. Sepet ve sipariş gibi kişisel yanıtlar yanlış ortak cache'e alınmayacak.

Doğrulama: Filtre ile facet sayıları aynı veri görünümünü temsil ediyor. Eski stok olayı son durumu geri almıyor. İndeks yeniden kurulurken arama erişilebilir kalıyor ve bitişte PostgreSQL ile örneklem karşılaştırması geçiyor. Kullanıcıya gösterilen stok ile satın alma anındaki kesin rezervasyon kararı ayrı sınanıyor.

14. 1000 kullanıcı hedefini doğru tanımlamak

Birbirine benzeyen dört sayı

Ölçü	Anlamı	Örnek
Kayıtlı hesap	Veritabanındaki toplam kullanıcı sayısı	10.000 hesabın çoğu o anda sitede olmayabilir.
Aktif sanal kullanıcı, VU	Senaryosunu sürdüren test oturumu	1000 VU'nun bir bölümü düşünme süresinde bekler.
Aynı anda işlemdeki istek	Henüz tamamlanmamış HTTP çağrısı	1000 VU varken sadece 40 istek sürüyor olabilir.
RPS	Bir saniyede işlenen HTTP istek sayısı	Bir kullanıcı yolculuğu birden çok istek üretir.

“1000 kullanıcıyla test ettim” cümlesinin yanına davranış, bekleme, süre ve gerçekleşen RPS yazılmalıdır. Örneğin tamamen varsayımsal olarak 1000 kullanıcı her 20 saniyede beş istek üretirse ortalama 250 RPS elde edilir. Gerçekte yanıt süreleri, think time ve hata yolları bu hesabı değiştirir. Bu sayı TechCart için ölçülmüş sonuç değildir.

Kararlı bir ölçüm penceresinde ortalama işlemdeki istek sayısı yaklaşık `RPS × ortalama yanıt süresi (saniye)` ilişkisinden değerlendirilebilir. Bu ilişki aktif kullanıcı sayısıyla doğrudan aynı değildir. Kullanıcı bekleme süreleri de vardır. p95'i ortalama yerine koyup kesin kapasite hesabı yapmak doğru olmaz.

Seçtiğimiz ana kullanıcı davranışı

Grup	Pay	1000 VU'da	Yolculuk
Ziyaretçi	%70	700	Listele, filtrele, ara, ürün detayını aç.
Sepet kullanıcısı	%20	200	Giriş yap, ürün ekle, miktar değiştir, sepeti kontrol et.
Satın alan	%10	100	Adres seç, sipariş oluştur, test ödemesini başlat, sonucu doğrula.

Adımlar arasında 2-8 saniye düşünme süresi olacak. Bu oran kullanıcı sayısının oranıdır; her grubun saniyedeki istek sayısı aynı olmayabilir. Satın alan grup daha fazla yazma çağrısı üretir. Bütün 1000 kullanıcının aynı anda ödeme istemesi ayrıca tanımlanan stres senaryosudur. Ana kapasite testinin sonucu o senaryoyu otomatik olarak kapsamaz.

Kapalı model ve açık model

Kapalı modelde belirli sayıda VU çalışır; her biri önceki yolculuğunu tamamlayıp yenisine geçer. Sistem yavaşlayınca tur sayısı ve RPS azalır. Bu model aktif kullanıcı deneyimi için uygundur fakat azalan geliş hızı sistemi olduğundan rahat gösterebilir. Bu etkiyi fark etmek için VU ile birlikte throughput ve süreler izlenir.

Açık modelde belirli hızda yeni iteration başlatılır; öncekinin bitmesi beklenmez. Böylece dışarıdan gelen sabit talebe sistemin yetişip yetişmediğini görürüz. k6 arrival-rate executor'ları iteration gelişini düzenler. Bir iteration beş HTTP çağrısı yapıyorsa 100 iteration/s, 100 RPS demek değildir. Tek istekli açık model örneğinde ikisi birbirine yaklaşır. k6 açık/kapalı modeller

Neyi kanıtlamayı hedefliyoruz?

Önce 50 eşzamanlı kullanıcının 30 dakikalık karma trafiğinde kabul hedeflerini sağlamak, sonra kaynak sınırları içinde sürdürülebilen kapasiteyi bulmak istiyoruz. 1000 seviyesine ulaşılamazsa rapor “başarısız proje” demez; hangi kaynakta hangi kapasitenin ölçüldüğünü gösterir. Sonraki karar bunun üzerine verilir: sorgu düzeltmek, tüketiciyi ayarlamak, cache eklemek veya kaynak bütçesini yeniden değerlendirmek.

Kaçınılacak hata: Bütün istekleri 429 ile reddederek sunucuyu hızlı göstermek kapasite değildir. Kabul edilen iş hacmi, iş doğruluğu ve kullanıcı gecikmesi birlikte değerlendirilir. Rate limit sonuçları raporda ayrıca görünür.

15. Sentetik veri, oturumlar ve ödeme simülasyonu

Neden bugünkü 3731 ürün yeterli başlangıç ama yeterli deney değil?

Mevcut ürün seti kurulum ve arama denemesi için yararlı. Fakat kullanıcı geçmişi, büyük sipariş tabloları ve tekrar üretilebilir stok dağılımı olmadan bazı sorguların büyüdükçe davranışını ölçemeyiz. Planlanan test seti yaklaşık 10.000 ürün, 10.000 sentetik kullanıcı ve 100.000 geçmiş siparişten oluşur. Bunlar başlangıç senaryo boyutlarıdır; “1000 kullanıcı için mutlaka bu kadar kayıt gerekir” kuralı değildir.

Sabit rastgelelik tohumu, aynı başlangıç verisini yeniden üretmeye yardımcı olur. Ürünlere kontrollü popülerlik dağılımı verilir: çok ziyaret edilenler, az ziyaret edilenler, stok dışı olanlar ve son birkaç adedi kalanlar. Geçmiş sipariş kalemleri, toplamlar ve stok ilişkileri tutarlı üretilir. Her kaydı rastgele bağlamak gerçekçi bir veritabanı oluşturmaz.

Çalışmaya özel veri yaşam döngüsü

Her veri yüklemesi bir `RunId`/veri seti kimliği taşır. Test hesapları ve oluşturulan siparişler bu kimlikle ilişkilendirilir. Temizlik yalnız o çalışmanın verilerini siler; tabloyu tamamen boşaltan genel komutlar kullanılmaz. Yük boyunca stok tüketileceği için test öncesi/sonrası durum kaydedilir. Cache karşılaştırmasının ikinci koşusu tükenmiş stokla yapılmaz; aynı başlangıç anına dönülür.

Hazırlık	Amaç	Kontrol
Ayrı LoadTest veritabanı, broker alanı ve indeks	Demo verisini ve mesajlarını korumak	Başlangıçta environment ve hedef adlarını doğrulama.
Sabit tohum ve veri manifest'i	Aynı deneyi tekrar kurmak	Ürün/kullanıcı/sipariş sayıları ve dağılımlar.
Kullanıcıya özel hesap, adres ve sepet	Kullanıcıların birbirini bozmasını önlemek	VU-hesap eşlemesinde tekillik.
RunId ile temizlik	Sadece test etkisini geri almak	Silinecek kayıtların ön sayımı ve sahipliği.

Sentetik hesaplar gerçek kişilerin e-posta, adres ve kart verilerini içermez. Örnek e-postalar `example.test` gibi ayrılmış bir alanla üretilebilir. Parolalar ve oturum token'ları rapora yazılmaz; fixture dosyaları erişimi kısıtlı test girdisidir.

Oturumlar uzun testte ne olacak?

Her sanal kullanıcı kendi oturumunu ve sepetini kullanacak. Tek token'ı bin VU'ya vermek aynı sepet üzerinde yapay yarış üretir. Bu, ayrı bir eşzamanlılık deneyi olarak yararlı olabilir ama normal müşteri modelimiz değildir. k6'daki test çapında VU kimliği fixture kullanıcısına eşlenir; çoklu yük üreticisinde kimlik aralıkları çakışmaz ve fixture kapasitesi kontrol edilir.

Mevcut JWT süresi ayarda 60 dakikadır. İki saatlik dayanıklılık testinde token dolacaktır. Kontrollü yeniden giriş veya o tarihte uygulanmış yenileme akışı gerekir. Bugün olmayan refresh endpoint'ini varmış gibi kullanmayız. Bütün VU'lar aynı saniyede yeniden giriş yapmasın diye süreye yayma uygulanır. Giriş yükü de raporda görünür; sonsuz 401 döngüsü test edilen alışveriş trafiğini değiştirmemelidir.

Ödeme sağlayıcısını yük üreticisine çevirmemek

Yoğun ödeme yükü için mevcut `IPaymentGateway` arayüzünün deterministik test adaptörü geliştirilecek. Adaptör, belirli girişte belirli başarı/ret/zaman aşımı sonucu ve kontrollü gecikme döndürür. Sağlayıcı token'ını taklit ederken uygulamanın deneme eşleme ve sonuç doğrulama kurallarını atlamaz. Callback tekrarları ve geç yanıt da üretilebilir.

Bu adaptör yalnız `LoadTest` ortamında açılabilir. Production'da test adaptörü seçilirse uygulama başlangıçta hata vermeli. Kamuya açık “ödemeyi başarılı yap” endpoint'i eklenmez. Demo Production ortamında gerçek iyzico sandbox adaptörünü kullanır. Sandbox ile az sayıda gerçek entegrasyon ve tarayıcı senaryosu ayrıca çalıştırılır; sağlayıcının sistemi 1000 VU testinin hedefi yapılmaz.

Yük üreticisi hedef VPS'te çalışmaz. Ayrı bilgisayarın CPU, RAM, ağ ve dosya tanımlayıcı sınırları da izlenir. Kaynak yetersizliği veya dropped iteration, hedef uygulamanın gerçek kapasitesi hakkında yanlış sonuç üretebilir. Cloud k6 hizmeti kullanmak zorunlu değildir.

Tamamlanma ölçütü: Aynı veri manifest'i yeniden yüklenebiliyor, her VU ayrı kimlikle çalışıyor, iki saatlik oturum davranışı tanımlı ve ödeme adaptörü Production'da etkinleşemiyor.

16. Yük testlerinin sırası ve uygulanması

Her test ayrı bir soruya cevap verir

Test	Başlangıç planı	Öğrenmek istediğimiz
Smoke	1-5 VU, yaklaşık 2 dakika	Script ve temel alışveriş yolu doğru mu?
İlk yayın kabulü	50 VU, 30 dakika sabit karma trafik	Demo için belirlenen kalite hedefleri sağlanıyor mu?
Kademeli kapasite	10 → 50 → 100 → 250 → 500 → 1000	Gecikme, hata veya birikim hangi basamakta artıyor?
Açık model	Tek okuma/iteration ile ayrı 25/50/100/200/400 geliş/s denemeleri	Hedeflenen istek gelişine gerçekten yetişiliyor mu?
Ani yük	50'den 1000'e yaklaşık 10 sn; 2 dk yük; 5 dk toparlanma	Kısa yoğunlukta sınırlar ve geri dönüş nasıl?
Stres	Satın alma payını ve sıcak ürün yoğunluğunu artır	Kapasite sınırı ve stok yarışları nasıl yönetiliyor?
Dayanıklılık	Ölçülen sürdürülebilir kapasitenin yaklaşık %70'inde 2 saat	Bellek, bağlantı, oturum ve kuyruk birikiyor mu?
Kesinti	Testte broker/cache/arama kes ve geri getir	Doğru veri korunup iş birikimi eritilebiliyor mu?

Bu süre ve hızlar başlangıç deney planıdır, ölçülmüş kapasite değildir. Smoke bozukken büyük teste geçilmez. İlk eşik sağlanmadan 1000 VU'ya çıkmak daha çok hatalı veri üretip nedenini bulmayı zorlaştırır. Ani yük seviyeleri de önceki denemelerde sistemi ve test verisini güvenli biçimde toparlayabildiğimizde uygulanır.

Basamakları nasıl okuyacağız?

Kademeli testte her seviyeye örneğin bir dakikalık artışla geçilir ve beş dakika sabit tutulur. Bu kısa ölçüm pencereleri ilk darboğazı bulur; sürdürülebilirlik iddiası için yeterli olmayabilir. Sonrasında aday kapasite daha uzun doğrulanır. İlk ısınma bölümü, kararlı bölüm ve toparlanma ayrı raporlanır. Bütün koşuyu tek p95'e indirgemek son basamaktaki bozulmayı gizleyebilir.

50 VU kabul testinin 30 dakikası sabit yük bölümüdür; ramp-up buna dahil sayılmaz. Üç grubu 35/10/5 kullanıcıyla çalıştırırız. 1000 seviyesinde 700/200/100 olur. Satın alma turunda adres seçimi, sipariş, test ödeme sonucu ve sipariş durumunu kontrol etme gerçekten uygulanmalıdır; yalnız ürün listesi açan 1000 VU'ya “eşzamanlı alışveriş” demeyiz.

Açık modelin özel kontrolleri

Hedeflenen iteration geliş hızı ile gerçekleşen hız yan yana gösterilir. `preAllocatedVUs` yetersizse veya işlem çok yavaşlarsa `dropped_iterations` görülebilir. Testi yalnız tamamlanan hızlı istekler üzerinden başarılı saymayız. Bir iteration tek HTTP isteği üretirse RPS daha kolay yorumlanır. Kullanıcı yolculuğu açık modelde kullanıldığında iteration/s ve RPS ayrı raporlanır. k6 constant-arrival-rate

Bu executor'da geliş hızını `sleep` ile ayarlamayız; bunu executor yapar. Kapalı modeldeki 2-8 saniye think time ise kullanıcı davranışının parçasıdır. İki yaklaşımın farklı amaçları vardır ve iki raporun VU sayıları bire bir kapasite karşılaştırması sayılmaz.

Durdurma ve toparlanma

Negatif stok, çift iş etkisi veya beklenmeyen gerçek sağlayıcı çağrısı görülürse test durdurulur ve veri korunarak incelenir. Bellek/disk sınırına yaklaşma veya sürekli büyüyen kuyrukta yük artışı kesilir. Uygulama veritabanını aşırı yüklemeye devam etmek yeni bilgi üretmeyebilir. Kuyrukların yük bittikten sonra normal seviyeye inmesi için ayrı toparlanma penceresi bırakılır.

Tamamlanma ölçütü: Her koşunun sorusu, başlangıç verisi, commit'i, süreleri, kabul eşikleri ve gerçek yükü kayıtlıdır. Testlerin türleri sadece dosya adı olarak bulunmaz; senaryonun davranışı o soruyu gerçekten sınar. k6 test türleri

17. k6 ile ölçülebilir senaryo yazmak

Önce mevcut okuma endpoint'iyle küçük bir örnek

Aşağıdaki örnek mevcut ürün listeleme sözleşmesine göre hazırlanmıştır. Yeni endpoint gerektirmez. **Bu belge çalışmasında script dosyası oluşturulmadı ve örnek çalıştırılmadı.** Uygulama aşamasında ayrı yük üreticisinde dosyaya kaydedilecek. Kodun amacı süreyi ölçmenin yanında cevap biçimini doğrulamak ve başarısız koşuda sıfırdan farklı çıkış üretmektir.

```
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate } from 'k6/metrics';

const base = (__ENV.BASE_URL || 'http://localhost:8081')
  .replace(/\/$/, '');
const unexpected = new Rate('unexpected_error_rate');

export const options = {
  vus: 5,
  duration: '2m',
  thresholds: {
    'http_req_duration{kind:read}': ['p(95)<500'],
    unexpected_error_rate: ['rate<0.01'],
    checks: ['rate==1'],
  },
};

export default function () {
  const res = http.get(
    `${base}/api/products?page=1&pageSize=20`,
    { timeout: '5s', tags: { name: 'catalog', kind: 'read' } }
  );
  let body = null;
  try { body = res.json(); } catch (_) { /* kontrol aşağıda */ }

  const valid = res.status === 200
    && Array.isArray(body?.items)
    && Number.isInteger(body?.pagination?.totalItems);
  check(res, {
    'katalog sözleşmesi doğru': () => valid,
    'sayfa sınırı korunuyor': () => valid && body.items.length <= 20,
  });
  unexpected.add(!valid);
  sleep(2 + Math.random() * 6);
}
```

Uygulama sırasında bu örnek örneğin `/Users/cagla/Desktop/TechCart/tests/load/catalog-smoke.js` dosyasına kaydedilebilir. Bu bir hedef dosya yoludur; belge üretimi bu dosyayı oluşturmaz. k6 ayrıca kurulmuş yük üreticisi makinede aşağıdaki komutla çalıştırılır. Yerel adres yalnız kendi bilgisayarındaki smoke denemesi içindir; VPS ölçümünde `BASE_URL` test sunucusunun erişilebilir adresi olur.

```
k6 version
k6 run \
  -e BASE_URL=http://localhost:8081 \
  /Users/cagla/Desktop/TechCart/tests/load/catalog-smoke.js
```

HTTP 200 olup HTML hata sayfası dönmesi de bu kontrolde başarısız olur. Yanıtın gerçekten boş olmaması gereken fixture senaryosunda `totalItems > 0` gibi veri koşulu da eklenir. Başka bir ortamda ürün sayısının tam 3731 olmasını zorunlu tutmayız; veri setinin manifest'ini kullanırız.

`check`, doğrulamanın sonucunu kaydeder. Tek başına her başarısız check k6 sürecini başarısız bitirmez; `threshold` geçme/kalma kuralını tanımlar. Bu örnekte bütün check'lerin geçmesi istendi. Gerçek akışta beklenen stok yetersizliği kendi check'iyle başarılı iş sonucu sayılabilir. k6 thresholds

Kullanıcı verisi ve tekrar anahtarı

Hedef fixture dosyasında kullanıcı kimliği, test parolası ve sahip olduğu adres kimliği bulunabilir. Büyük dosya her VU için tekrar kopyalanmasın diye k6 `SharedArray` init aşamasında kullanılır; dosya okuma ve JSON ayrıştırma onun callback'i içinde yapılır. Oturum gibi değişken VU durumu bu salt okunur ortak diziye yazılmaz. k6 SharedArray

Tek yük üreticili ilk sürümde `exec.vu.idInTest - 1` ile kullanıcı seçilebilir; indeks yoksa test açık hata vermelidir. Dağıtık çalıştırmada kimliklerde boşluk olabileceği için test çapında eşleme/ayrılmış hesap aralığı hazırlanır. Aynı hesabı iki VU'ya sessizce vermeyiz. k6 execution kimlikleri

```
// HEDEF: users.json üretilince tek üreticili script'e eklenir.
import { SharedArray } from 'k6/data';
import exec from 'k6/execution';
const users = new SharedArray('test-users', () =>
  JSON.parse(open('./fixtures/users.json'))
);
function currentFixture() {
  const user = users[exec.vu.idInTest - 1];
  if (!user) throw new Error('VU için ayrı fixture bulunamadı');
  return user;
}
```

Her satın alma turu için örneğin `RunId + VU kimliği + tur numarası` ile yeni iş niyeti anahtarı üretilebilir. Aynı turun ağ retry'ında bu anahtar korunur. Başka tur yeni anahtar alır. Token süresi dolduğunda yeniden giriş yapılır; bir siparişin retry'ı yeni tur sayılıp çift sipariş üretilmez. Cookie/CSRF geçişinden sonra k6 kendi cookie jar'ı ve CSRF token'ıyla yeni tarayıcı sözleşmesini izler; eski bearer-token script'i değişmeden kullanılamaz.

Karma senaryo iskeleti: hedef uygulama

Aşağıdaki iskeletteki `journeys.js` henüz projede yoktur. `browse`, `cart` ve `buy` fonksiyonları; gerçek endpoint sözleşmeleri, fixture ve LoadTest ödeme adaptörü tamamlandıktan sonra yazılacaktır. Bu örnek 1000 VU dağılımını gösterir; tek başına çalışabilir bir yük testi değildir.

```
// HEDEF UYGULAMA: journey yardımcıları henüz oluşturulmadı.
import { browse, cart, buy } from './journeys.js';
export { browse, cart, buy };

export const options = {
  scenarios: {
    visitors: {
      executor: 'constant-vus', exec: 'browse',
      vus: 700, duration: '30m',
    },
    cart_users: {
      executor: 'constant-vus', exec: 'cart',
      vus: 200, duration: '30m',
    },
    buyers: {
      executor: 'constant-vus', exec: 'buy',
      vus: 100, duration: '30m',
    },
  },
};
```

İlk yayın kabulünde aynı yapı 35/10/5 olur. Kademeli testte üç senaryonun `ramping-vus` aşamaları aynı zaman çizelgesinde kendi oranlarına göre hazırlanır. Hızlıca 1000 ile başlamak yerine önce smoke ve 50 VU kapısı tamamlanır.

Satın alma yolculuğu sırası: oturumu hazırla → sepeti istenen duruma getir → fiyat/miktarı kontrol et → kendi adresinle sipariş oluştur → aynı niyet anahtarıyla checkout başlat → test sağlayıcısının sonucunu üret → yetkili sipariş API'sinden nihai durumu doğrula. Bugün olmayan sipariş detay veya test callback endpoint'i bu metin nedeniyle var kabul edilmez. Uygulama aşamasında sözleşme eklenir ya da mevcut listeleme sonucu güvenli biçimde kullanılır.

Teknik hata ile beklenen iş sonucunu ayırmak

5xx, bağlantı hatası, timeout, bozuk cevap ve geçersiz durum geçişi teknik hata sayılır. Beklenen “stok yetersiz” sonucu ise ancak tanımlı HTTP durumu ve tanımlı iş hata kodu birlikte eşleşirse ayrı sayılır. Bütün 400/409 cevaplarını beklenen kabul etmek, doğrulama ve idempotency hatalarını gizler. 401 normal alışverişte otomatik başarı sayılmaz; oturum yenileme sonucu ayrıca ölçülür.

Özel `unexpected_error_rate` her iş isteği için bir örnek alır. Dahili yazma ve dış sağlayıcı süreleri `kind=write_internal` ve `kind=provider` gibi sınırlı etiketlerle ayrılır. Toplam checkout süresi de kullanıcı deneyimi olarak korunur; dış çağrıyı ayırmak toplam beklemeyi rapordan silmek değildir.

Açık model için seçenek örneği

```
// Tek HTTP isteği yapan fonksiyona uygulanacak hedef seçenekler.
const scenario = {
  executor: 'constant-arrival-rate',
  rate: 100,
  timeUnit: '1s',
  duration: '5m',
  preAllocatedVUs: 50,
  maxVUs: 200,
};
// Geliş hızı executor'dan gelir; sona pacing sleep eklenmez.
// dropped_iterations için ayrıca count==0 eşiği konur.
```

Buradaki 100 geliş/s ve VU sınırları deneme değerleridir. İşlem süresi uzadığında daha fazla VU gerekebilir; yük üreticisi kapasitesi kontrol edilmeden sınır arttırılmaz. Testin beklediği yükün verilememesi, tamamlanan istekler hızlı olsa da geçer sonuç değildir.

Sonuç okunurken önce threshold geçme/kalma durumu ve süreç çıkış kodu, ardından checks, `http_reqs`/gerçekleşen hız, endpoint süreleri ve özel hata oranı incelenir. Açık modelde `dropped_iterations`, karma modelde tamamlanan satın alma sayısı da rapora alınır. Sonra aynı zaman aralığı sunucu panelleriyle eşleştirilir. k6 özeti tek başına veritabanı doğruluğunu kanıtlamaz; son koşul sorguları ayrıca çalışır.

Uygulama ve doğrulama: Script ayrı makinede çalıştırılacak; özet JSON ve çalışma manifest'i saklanacak. Özet dışa aktarımı, seçilen k6 sürümünün `handleSummary` sözleşmesiyle dosyaya yazılabilir; dosyanın oluştuğu ve hata çıkış kodunun korunduğu smoke'ta kontrol edilir. Ham her istek kaydını sınırsız JSON'a yazmak disk ve üretici yükü yaratabilir. Gerekli özet ve zaman serisi seçilir. Bu rehberde hiçbir örnek için ölçüm sonucu uydurulmamıştır.

18. Hata deneyleri ve doğruluk testleri

Yük testi ile hata deneyi aynı şey değildir

Kapasite testi normal trafiğin sınırını arar. Hata deneyi belirli bir bozulmada doğru davranışın korunup korunmadığını sorar. İkisini önce ayrı çalıştırmak nedeni bulmayı kolaylaştırır. Sonra seçilmiş yük altında kesinti deneyleri eklenir. Deneyler yalnız izole test ortamında yapılır; demo kullanıcıları etkilenmez.

Deney	Nasıl sınanır?	Beklenen kanıt
Stok 10, 100 alıcı	Farklı hesaplar aynı ürüne aynı anda bir adet ister	Toplam ayrılan/satılan miktar sınırı aşmaz, negatif stok yok.
Aynı callback'in tekrarı	Aynı kalıcı denemeye ait sonuç paralel gönderilir	Tek ödeme ve tek stok etkisi.
Başarı ile süre sonu yarışı	Aynı sipariş için ödeme ve Worker geçişi birlikte tetiklenir	Geçerli tek durum yolu, tahsilat gerçeği korunur.
Commit sonrası süreç kapanması	İş verisi/outbox commit'inden sonra teslimattan önce süreç durdurulur	Açılışta bekleyen olay teslim edilir.
RabbitMQ kapalı	Sipariş kaydı sırasında broker erişimi kesilir	Outbox kalır, broker dönüşünde birikim erir.
Redis kapalı	Katalog okunurken cache erişimi kesilir	Sınırlı fallback, checkout doğruluğu değişmez.
Elasticsearch kapalı	Arama ve katalog ayrı çağrılır	Arama kontrollü hata; katalog ve sipariş devam edebilir.
Eski olay geç gelir	Sürüm 42'den sonra 41 uygulanmaya çalışılır	Görünüm 42'de kalır.
Yetkisiz nesne erişimi	Kullanıcı A, B'nin adres/sipariş kimliğini gönderir	Veri açığa çıkmaz, iş oluşmaz.

Deney	Nasıl sınanır?	Beklenen kanıt
Yedek ve sürüm dönüşü	Ayrı kurulumda restore ve eski uyumlu imaj denenir	Süreler ölçülür, veriler ve temel akış doğrulanır.

Süreç kapatma deneyi sadece `publish` çağrısının önüne rastgele hata koymak değildir. Seçilen kesinti noktası commit'in gerçekten sonrasında olmalıdır. Teste özel hata enjeksiyon noktası oluşturulursa Production'da erişilememeli ve iş kodunun davranışını değiştirmemelidir.

Veritabanında neyi kontrol edeceğiz?

Test başlamadan stok, rezervasyon ve ilgili kayıtların başlangıç anı alınır. Sonrasında test RunId'sine göre sipariş/ödeme sayıları, kalem toplamları, aktif rezervasyonlar ve stok değişimleri karşılaştırılır. Her başarılı ödeme için beklenen sipariş ilişkisi, her iptal için serbest bırakma kontrol edilir. İş kuralındaki vergi/kargo yuvarlamaları da toplamların parçasıdır.

Sipariş/ürün başına benzersiz rezervasyon, idempotency kayıtlarının tekilliği ve ödeme sağlayıcı referanslarının tekrar etkisi test edilir. Bunlar henüz eklenecek şema alanları olduğundan bugünkü veritabanında bu doğrulamaların hazır SQL'i bulunduğu iddia edilmez. Negatif stok taraması yararlıdır ama yeterli değildir: stok pozitif kalırken iki kez gereksiz düşmüş de olabilir.

Hangi test katmanı neyi yapar?

Birim testleri, para hesabı, rezervasyon durumu ve izinli geçiş gibi saf kuralları hızlı sınar. **Entegrasyon testleri**, Testcontainers ile gerçek PostgreSQL/RabbitMQ/Redis/Elasticsearch davranışını ve transaction sınırını doğrular. Her test bütün servisleri açmak zorunda değildir; ilgili bağımlılık seçilir. **Mimari testler**, Domain'in Infrastructure'a bağlanmamasını ve değiştirilen modüllerin Contracts üzerinden iletişimini kontrol eder. Testcontainers .NET

Playwright, tarayıcıda kayıt/giriş, sepet, adres, checkout dönüşü ve sonuç sayfasını doğrular. **k6**, HTTP düzeyinde çok sayıda kullanıcı ve süre davranışını üretir. 1000 tam tarayıcı açmak ilk kapasite yaklaşımımız değildir. Az sayıda browser testi kullanıcı arayüzünü, k6 ise backend yükünü kapsar. Playwright belgeleri

Tamamlanma ölçütü: Hata sırasında yanlış veri oluşmuyor; kesinti bittikten sonra sistem kendi birikimini eritiyor. Sadece exception çıkmaması yeterli değil. Her deneyin başlangıç durumu, kesinti noktası, beklenen veri sonucu ve gerçek sonucu kayıtlıdır.

19. Raporlama ve kapasite kararları

İlk yayın için kabul hedefleri

Ölçüm	Hedef	Nasıl değerlendirilir?
Okuma endpoint'leri p95	500 ms altında	Endpoint/grup bazında; ısınma ve sabit yük ayrı.
Dahili yazma p95	1 saniye altında	Dış sağlayıcı beklemesi ayrı etiketlenir.
Beklenmeyen hata	%1 altında	İş sonucu olmayan teknik hatalar; payda ve sınıf tanımlı.
Projeksiyon gecikmesi p95	Normal çalışmada 5 saniye altında	İşlenen olay histogram'ı ve bekleyen en eski iş birlikte.
Veri doğruluğu	Çift sipariş/çift stok etkisi/negatif stok 0	Veritabanı son koşulları ve eşzamanlılık deneyleri.
İş birikimi	Sürekli büyüyen birikim yok	Yükte eğim, yük bitince toparlanma süresi.

p95, isteklerin %95'inin altında kaldığı süredir; bütün isteklerin hızlı olduğu anlamına gelmez. p99 kuyruktaki daha yavaş kullanıcıları gösterir. Yalnız ortalama süre birkaç çok yavaş isteği gizleyebilir. Tüm endpoint'leri birleştiren tek histogram da az kullanılan yavaş bir yazma işlemini çok sayıdaki hızlı okumayla örtebilir.

Bu hedefler ilk yayın için mühendislik kabul koşullarıdır; sözleşmesel yüksek erişilebilirlik garantisi değildir. Dış sağlayıcının süresi ayrı ölçülür ama kullanıcının toplam checkout bekleyişi de raporda kalır. Beklenen iş reddi artarsa bunun nedeni incelenir; teknik hata oranından ayrı olması müşteri etkisinin önemsiz olduğu anlamına gelmez.

Her koşuda doldurulacak rapor

Alan	Kaydedilecek bilgi
Kimlik	RunId, tarih/saat, commit SHA, API/Worker/frontend imaj etiketleri.
Ortam	Environment, VPS CPU/RAM/disk, servis sınırları, ağ bölgesi, yük üreticisi özellikleri.
Veri	Seed, veri manifest sürümü, ürün/kullanıcı/geçmiş sipariş sayıları, başlangıç stok dağılımı.
Politikalar	Cache kapalı/soğuk/sıcak, TTL, tüketici eşzamanlılığı, havuz sınırları, trace sampling.
Yük	VU ve davranış dağılımı, süre, hedef iteration hızı, gerçekleşen RPS, dropped iteration.
Sonuç	Endpoint bazında p50/p95/p99, teknik hata, iş reddi, 401/429, timeout, tamamlanan alışveriş sayısı.
Kaynaklar	CPU/RAM/GC, DB süre/bekleme, Redis hit/miss, kuyruk/outbox/projeksiyon gecikmesi.
Doğruluk	Ön/son veri karşılaştırması, tekrar etkileri ve negatif stok denetimi.
Karar	Geçti/kaldı, darboğaz kanıtı, sonraki tek değişiklik ve tekrar ölçüm koşulu.

Raporun sonunda “1000 kullanıcı başarılı” yerine şu biçim kullanılır: “Belirtilen kaynak, veri seti, davranış ve süre içinde N aktif kullanıcıda kabul koşulları sağlandı; M seviyesinde şu sınır görüldü.” Henüz N ve M için TechCart'a ait ölçüm yoktur. Örnek tabloyu sayı uydurarak doldurmayacağız.

Darboğaza göre karar verme

DB sorguları ve CPU artıyorsa önce plan/indeks ve yinelenen okuma incelenir. DB bağlantı beklemesi yüksekse havuz ve uzun transaction araştırılır. API CPU'su dolu fakat DB sakinse serialize, hesaplama, GC ve thread pool izlerine bakılır. Kuyruk sürekli büyüyorsa tüketici süresi, retry döngüsü ve bağımlılık kapasitesi kontrol edilir. Yavaşlık yalnız dış ödemedeyse katalog cache'i çözüm sayılmaz.

Bir değişkeni değiştirip aynı koşuda tekrar ölçmek nedenselliği görmeyi sağlar. Aynı anda cache, indeks, sunucu ve script'i değiştirirsek kazancın nereden geldiğini açıklayamayız. Ölçüm gürültüsü varsa aday sonuç tekrarlanır; aynı başarılı testi gerekçesiz sürekli çalıştırmak da öğrenme sağlamaz.

Tamamlanma ölçütü: Her optimizasyonun önce/sonra raporu var. Kapasite ifadesinin koşulları açık, veri doğruluğu kanıtı bulunuyor ve test üreticisi yetersizliği hedef uygulama kapasitesi diye sunulmuyor.

20. İnternete güvenli açılmak

Tek giriş noktası ve özel servis ağı

İlk yayın tek Linux VPS ve Docker Compose üzerinde olacak. Caddy, alan adında HTTPS sonlandırıp `/api` isteklerini API'ye, diğerlerini Next.js'e yönlendirecek. Aynı origin; protokol, alan adı ve portun ortak olmasıdır. Bu yaklaşım tarayıcı yapılandırmasını ve cookie kullanımını sadeleştirir. Sertifika alımı/yenilemesi için DNS ve gerekli ağ erişimleri doğru olmalıdır. Caddy otomatik HTTPS

PostgreSQL, Redis, RabbitMQ yönetimi, Elasticsearch ve telemetri uçları internete açılmayacak. Yönetim gerekiyorsa özel erişim veya SSH tüneli kullanılır. Bugünkü localhost geliştirme portları yayın şablonu diye kopyalanmaz. Güvenlik duvarı, SSH anahtarı, kalıcı disk ve container kullanıcı yetkileri deployment işinin parçasıdır.

Frontend hedefte göreli `/api` kullanacak. Mevcut Docker build'i boş olmayan public API URL'si bekleyebildiği için yalnız değeri boş bırakmak yeterli düzenleme olmayabilir. Build doğrulaması ve API client bu hedefe bilinçli biçimde uyarlanacak. Public URL tarayıcıya görünür; secret değildir. Ödeme callback'i sağlayıcının/tarayıcının erişebileceği API URL'sini, sonuç yönlendirmesi frontend URL'sini kullanmalıdır. Container içi `api:8080` adresi dış dünyaya verilmez.

Oturum: bugünden hedefe

Bugün frontend JWT'yi localStorage üzerinden taşıyor. Hedefte token HttpOnly, Secure, SameSite=Lax cookie'de taşınacak. HttpOnly, JavaScript'in token'ı doğrudan okumasını zorlaştırır; XSS riskini tamamen kaldırmaz. Secure cookie HTTPS gerektirir. Tarayıcı cookie'yi otomatik gönderdiği için yazma isteklerinde CSRF koruması gerekir.

CSRF, başka bir sitenin kullanıcının mevcut oturumuyla istek yaptırabilmesidir. Frontend sunucudan aldığı antiforgery token'ını yazma isteğinde uygun header ile gönderir; sunucu doğrular. CORS ile CSRF aynı savunma değildir. SameSite tek başına bütün yazma güvenliği yerine kullanılmaz. ASP.NET Core CSRF koruması

Sağlayıcı callback'i kullanıcı tarayıcısının CSRF token'ını taşımayabilir; sağlayıcı token'ını sunucudan doğrulayan ayrı akış olarak tasarlanır. Bu istisna bütün ödeme endpoint'lerine sınırsız erişim vermek değildir. Özellikle cross-site POST'ta SameSite=Lax oturum cookie'sine güvenilmez. Callback iş sonucunu kalıcı checkout denemesiyle eşlemeli; normal kullanıcı sahipliğiyle sipariş sorgulama ayrı korunmalıdır.

Logout yalnız tarayıcıdaki token'ı silmekten ibaret değildir. Mevcut revoked-token yaklaşımıyla sunucunun iptali kontrol etmesi ve parola değişiminde gerekli oturumların geçersizleşmesi doğrulanacak. JWT kullanmak sunucuda hiç oturum kontrolü gerekmediği anlamına gelmez.

Yetki, rate limit ve gizli değerler

Her adres ve sipariş erişiminde nesnenin oturumdaki kullanıcıya ait olduğu kontrol edilir. Başkasının kimliğini bilen kişi o kaydı okuyamaz veya onunla sipariş veremez. Bu kontrol UI'da buton gizlemekle sağlanmaz; API kullanım senaryosunda uygulanır.

Giriş/kayıt, katalog okuma ve kullanıcı bazlı yazma için ayrı rate limit politikaları seçilir. Reverse proxy arkasında istemci IP'si yalnız güvenilen proxy başlıklarından alınır. Rate limit saldırı ve yanlış kullanım sınırıdır; beklenen 50/1000 kullanıcı trafiğini ölçmeden rastgele dar bir sınır koymak yük testini anlamsızlaştırır. 429 sonuçları teknik başarı gibi gizlenmez.

`.env` ve User Secrets Git dışında kalır; bu dosyalar şifreli kasa değildir. Sunucuda erişim sınırlandırılır, CI secret'ları loga yazdırılmaz. Anahtar daha önce Git'e girdiyse güncel dosyadan silmek eski kopyayı yok etmez;

anahtar yenileme gerekir. Sandbox anahtarları dahi kişisel hesap bilgisi olarak korunur. Yedeklerde ve hata çıktı paketlerinde de aynı kontrol yapılır.

Tamamlanma ölçütü: HTTPS, cookie/CSRF, nesne sahipliği, logout/parola değişimi ve rate limit testleri geçiyor. Açık servis portları beklenenlerle sınırlı. Arayüzde sandbox demo bilgisi görünür; gerçek kart/tahsilat beklenmediği anlaşılır.

21. Otomatik doğrulama, dağıtım ve geri dönüş

CI hangi güveni sağlayacak?

CI, kod her değiştiğinde aynı kontrollerin temiz ortamda çalıştırılmasıdır. GitHub Actions backend restore/build/test, frontend lint/type/build ve küçük uçtan uca akışları çalıştıracak. Mevcut birim testlerinin üzerine kritik entegrasyon ve mimari testleri eklenecek. Testcontainers için runner'da Docker erişimi gerekecek. Önceki lint borcu kapatılmadan kontrol başarısızlığını görmezden gelen bir yayın hattı kurulmayacak. GitHub Actions .NET rehberi

Her commit'te iki saatlik yük testi çalıştırmak ilk hedef değildir. Kısa smoke/regresyon testleri sık, uzun kapasite ve kesinti testleri planlı çalışır. Gerçek sandbox entegrasyonu ayrı ve sınırlı tutulur. Testte kullanılan deterministik ödeme adaptörü, gerçek sağlayıcıyla sözleşmenin çalıştığını tek başına kanıtlamaz.

İmaj ve migration disiplini

İmajlar commit SHA ile etiketlenir; “latest” hangi kodun çalıştığını anlamak için yeterli değildir. Aynı doğrulanmış imaj terfi ettirilir. SDK, temel imaj, paket ve lock dosyaları tekrarlanabilirliği artırır; yalnız doğrudan paketleri sabitlemek bütün geçişli bağımlılıkları ve imaj içeriğini değişmez yapmaz. Güvenlik güncellemeleri için kontrollü yenileme gerekir.

Dağıtım sırası: geçerli yedek oluştur ve doğrula → migration'ı tek kontrollü işlem olarak uygula → yeni API/Worker/frontend imajlarını başlat → readiness kontrol et → kısa smoke test yap → sonuç ve sürümü kaydet. Her API kopyasının açılışta aynı anda migration çalıştırması yerine tek migrator/komut kullanılır. Temiz kurulum betiği gelecekte ortam bağımsız dağıtım aracına dönüştürülecekse hedef korumaları korunmalıdır.

Liveness ve readiness

Liveness sürecin cevap verebildiğini gösterir. Readiness, yeni iş almaya yeterince hazır olup olmadığını gösterir. PostgreSQL'in çalışmaması temel yazma akışını etkiler. Redis'in çalışmaması ise fallback tasarımında bütün API'yi kapatmak zorunda değildir. Elasticsearch araması bozulmuşken katalog/sipariş hazır kalabilir. Alt bileşen durumları panelde “degraded” olarak görülebilir.

Yalnız process'in Up görünmesi yeterli değildir. Uygulamanın hazır olduğuna dair endpoint ve ardından gerçek bir hafif katalog/oturum smoke kontrolü gerekir. Sağlık endpoint'leri iç bağlantı/parola/exception ayrıntısını herkese açmamalıdır. Broker kesintisinin readiness etkisi outbox politikasıyla birlikte seçilir; outbox varsa kısa kesintide yerel iş kabulü devam edebilir.

Eski imaja dönmek neyi geri alır?

Eski imajı başlatmak kodu geri alır; veritabanı şemasını ve yeni sürümün yazdığı veriyi otomatik geri almaz. Bu yüzden önce genişlet, sonra eski alanı kaldır yaklaşımı uygulanır. Örneğin yeni nullable sütun ekleme eski sürümle uyumlu olabilir; aynı dağıtımda eski sütunu silmek geri dönüşü bozabilir. Veri dönüşümü ve destructive migration ayrı değerlendirilir.

Deployment smoke başarısız olursa eski uyumlu imaj etiketine dönülür ve yeniden doğrulanır. Veritabanını yedekten geri almak rutin rollback yerine kullanılmaz; yedek sonrasındaki kullanıcı verisini kaybettirebilir. Geri dönüş komutu, hangi şema sürümleriyle güvenli olduğu ve doğrulama adımı yazılı olacak.

Tamamlanma ölçütü: CI temiz runner'da geçiyor; commit'ten çalışan imaja iz var. Bozuk sürüm deneyiyle sağlık/smoke başarısızlığı fark ediliyor ve önceden doğrulanmış uyumlu sürüme dönülebiliyor. Bu çalışma doküman üretimi olduğu için burada deployment yapılmamıştır.

22. Yedek, geri yükleme ve işletim

Volume ile yedeği ayırmak

PostgreSQL volume'u container yenilendiğinde veriyi korur. Aynı sunucunun diski bozulursa bu koruma yeterli değildir. Yedek bağımsız bir kopyadır; sunucu dışında saklanır. Günlük, şifreli PostgreSQL yedeği ilk hedeftir. Veritabanı dump'ı alınabilir, ardından şifreleyerek başka sağlayıcıdaki nesne depoya veya yedek hizmetine aktarılabilir. Şifreleme anahtarının da kaybolmayacak ve yedek dosyayla aynı açıklıkta olmayacak şekilde saklanması gerekir.

PostgreSQL için SQL dump, dosya sistemi yedeği ve sürekli arşivleme farklı yaklaşımlardır. İlk demo için mantıksal dump daha basit bir başlangıç olabilir. `pg_dump` custom format ve `pg_restore` seçimi geri yükleme prosedürüyle birlikte doğrulanacak. Veritabanı rolleri, yetkiler ve gerekli extension'lar da kurulum planının parçasıdır. PostgreSQL 17 yedek yaklaşımı, pg_dump

RPO ve RTO ne anlama geliyor?

RPO 24 saat, felakette kabul edilebilecek veri kaybı penceresinin başlangıç hedefidir. Her gün başarılı yedek alındığında bu hedefe yaklaşılır; başarısız yedek iki gün fark edilmezse sağlanmaz. **RTO 2 saat**, hizmeti geri getirmek için hedeflenen süredir. Yeni sunucu, kurulum, indirme, restore ve doğrulama birlikte bu süreye dahildir. Bunlar henüz ölçülmüş hizmet garantisi değildir.

Gerçek satışta 24 saat ödeme/sipariş kaybı kabul edilemeyebilir. Daha sık yedek veya WAL/PITR yaklaşımı gerekebilir. Bu değerlendirme gerçek tahsilat kapısında yeniden yapılır; demo hedefi sessizce gerçek mağazanın taahhüdüne dönüşmez.

Geri yükleme tatbikatı

1. Başlangıç zamanı kaydedilir; ayrı, boş hedef kurulur.
2. Sunucu dışındaki yedek indirilir, bütünlük ve şifre çözme doğrulanır.
3. PostgreSQL ve gerekli roller/uzantılar hazırlanır; veri geri yüklenir.
4. Şema sürümleri, önemli kayıt sayıları ve ilişkiler kontrol edilir.
5. API/Worker uyumlu imajlarla açılır; giriş, katalog ve örnek sipariş sorgulanır.
6. Elasticsearch esas veriden yeniden kurulur; olay teslimatı ve projeksiyon kontrol edilir.
7. Bitiş zamanı ve eksik adımlar kaydedilir; RPO/RTO sonucu raporlanır.

Restore'u mevcut demo veritabanının üzerine denemeyiz. Sadece yedek dosyasının oluşması da geri dönüş kanıtı değildir. Tatbikatta yedekten açılan uygulama temel iş akışını çalıştırmalıdır. pg_restore

Hangi veriyi nasıl geri getiririz?

PostgreSQL iş verisi, kalıcı outbox ve inbox kayıtları korunmalıdır. Redis cache yeniden üretilebilir; öncelikli yedek değildir. Elasticsearch de esas veriden ve sürümlü olaylardan yeniden kurulabilir. RabbitMQ kuyruğu, kalıcı iş kayıtları ve outbox ile birlikte değerlendirilir; broker'ı silince her işin kendiliğinden geri geleceği varsayılmaz. Teslim edildi işaretli fakat henüz tamamlanmamış mesajların toparlanma/uzlaştırma prosedürü gerekir.

HTTPS sertifika deposu ve cookie korumasında kullanılan Data Protection anahtarları gibi dosyaların kalıcılığı da planlanır. Anahtarlar değişirse mevcut oturumlar geçersizleşebilir. Bu, PostgreSQL dump'ının bütün sistem

durumunu kapsamadığını gösterir. Hangi sırrın/anahtarın yeniden üretileceği, hangisinin korunacağı yazılı olmalıdır.

Günlük işletim

Disk doluluğu, başarısız yedek, artan outbox yaşı ve hata kuyruğu için uygulanabilir uyarılar tanımlanır. Logların container diskini sınırsız büyütmemesi için rotasyon ayarlanır. İşletim notunda “arama çalışmıyor”, “ödeme sonucu bilinmiyor” ve “siparişler birikiyor” olayları için ilk bakılacak paneller ve toparlanma adımları bulunur.

Tamamlanma ölçütü: Sunucu dışındaki şifreli yedekten temiz kurulum açılmış, süre ölçülmüş ve veri/iş akışı doğrulanmış olur. Tek sunuculu başlangıcın yüksek erişilebilirlik sağlamadığı açık kalır; yedek ve hızlı geri dönüş bu riski azaltır, ortadan kaldırmaz.

23. Bütçe, yol haritası ve yayın kapıları

Kaynak seçimi bir hipotezdir

İlk kaynak profili 4 vCPU/8 GB RAM olarak düşünülüyor. PostgreSQL, Elasticsearch, RabbitMQ, Redis, API, Worker ve frontend aynı makineyi paylaşacak. Özellikle Elasticsearch heap'i, PostgreSQL belleği ve işletim sistemi dosya cache'i birlikte değerlendirilir. Her container'a toplam RAM kadar limit vermek gerçek bir kaynak planı değildir. Disk IOPS ve ağ da yalnız vCPU kadar belirleyici olabilir.

Aşağıdaki tablo sağlayıcı fiyat teklifi değildir; **30 USD toplam sınırını bölmek için hedef bütçe** örneğidir. Uygun bir 4 vCPU/8 GB teklif bulunacağı garanti edilmez. Yayın anında vergi, kur, bölge ve yenileme fiyatı dahil güncel teklif kontrol edilir. Bütçe tutmazsa ücretsizmiş gibi ek servis yazmak yerine kaynak profili veya yayın kapsamı yeniden değerlendirilir.

Kalem	Aylık hedef pay	Not
VPS	20 USD	Hipotez; güncel teklif ile doğrulanacak.
Sunucu dışı yedek	3 USD	Saklama ve indirme/geri yükleme bedeli birlikte.
Alan adı	1 USD	Yıllık ücretin aylık karşılığı; yenileme farkı kontrol edilir.
Vergi/kur/ağ farkı	4 USD	Yerel koşullara göre değişir.
Pay	2 USD	Küçük sapma rezervi.
Toplam üst hedef	30 USD	Ücretli gözlemleme ve ek sunucu varsayılmadı.

Yük üreticisi hedef VPS'ten ayrı olacak. Kendi bilgisayarını kullanmak başlangıç maliyetini azaltabilir; kiralanan üreticinin çalışma saatleri de bu bütçeye dahil edilir. Staging veya LoadTest'i demo ile aynı kaynaklarda aynı anda çalıştırmak performans izolasyonu sağlamaz. Ayrı kaynak veya zamanlanmış geçici test kurulumu seçilir; kamuya açık demo üzerinde yük üretilmez.

Tamamlanan kurulumdan sonra uygulama sırası

1. Gözlemlenebilirlik. API/Worker JSON logları, OTel izleri, metrikler, Alloy ve ilk panel eklenir. Sağlık uçları ve veri gizleme kapsamı belirlenir. Kontrollü bir hata aynı iz üzerinden bulunabiliyorsa aşama kapanır. Böylece sonraki geliştirmelerin etkisini görebiliriz.

2. Sipariş ve ödeme doğruluğu. Ortak transaction, siparişe bağlı rezervasyon, stok kısıtları, kalıcı CheckoutAttempt ve idempotency tamamlanır. Kısa transaction ve dış sağlayıcı ayrımı korunur. Paralel stok, callback tekrarı, ödeme/iptal yarışı ve yarım işlem testleri geçmeden performans başarısı ilan edilmez.

3. Güvenilir olay teslimatı. Kalıcı outbox/inbox, sürümlü olaylar ve Worker tüketicileri eklenir. Hata kuyruğu ve yeniden işleme prosedürü yazılır. Broker kesintisi ve commit sonrası kapanma deneyinde işler kaybolmuyorsa kapanır.

4. Başlangıç kapasitesi. Deterministik veri seti, LoadTest ödeme adaptörü ve k6 kullanıcı yolculukları hazırlanır. Cache eklenmeden smoke, 50 VU kabulü ve ilk basamaklar ölçülür. Bu ölçüm sonraki iyileştirmenin karşılaştırma noktasıdır; sistem henüz eşikleri sağlamıyorsa rapor bunu açık söyler.

5. Darboğazı iyileştirme. Önce sorgu planları, indeks ve bağlantı sınırları düzenlenir. Tekrarlanan katalog yükünde ölçülen fayda varsa Redis eklenir. Arama/facet tutarlılığı ve alias geçişi tamamlanır. Her değişiklikten sonra aynı deneyle yeniden ölçülür; stok/ödeme doğruluğu regresyonu da çalışır.

6. Kontrollü demo yayını. HTTPS, cookie/CSRF, yetkilendirme ve rate limit tamamlanır. CI/CD, geri dönüş ve geri yükleme tatbikatı geçer. 50 VU/30 dakika kabul hedefleri sağlanınca sandbox demo internete açılır. 1000 kullanıcı basamakları ve dayanıklılık çalışmaları sonraki döngülerde devam edebilir.

Haftada 8-12 saatle bu, birkaç aylık gelişim programıdır. Her aşama bir haftaya sığmak zorunda değildir; özellikle transaction/ödeme ve hata testleri birden fazla çalışma dönemi ister. Tamamlanma takvim tarihine değil kanıta bağlanır. Aynı anda bütün araçları eklemek yerine bir aşamanın karar ve doğrulaması bitirilir.

Her aşamadan dört somut çıktı

1. **Karar notu:** Sorun, seçenekler, seçilen yaklaşım ve bedeli. Örneğin “neden cache-aside?”
2. **İncelenebilir değişiklik:** İlgili dosyalar ve dar kapsamlı uygulama; henüz yalnız bu belge üretildi.
3. **Doğrulama:** Otomatik test veya kontrollü hata deneyi ve çalıştırma komutu.
4. **Kanıt raporu:** Sürüm, koşullar ve gözlenen sonuç; hedef ile sonuç ayrı.

Gerçek satış için ayrı kapı

Sandbox demo; gerçek sağlayıcı hesabı, gerçek tahsilat, iade/iptal ve uzlaştırma operasyonunun yerine geçmez. Gerçek satışa geçmeden hesap kurtarma, kişisel veri saklama/silme, teslimat ve destek süreçleri, dolandırıcılık/itiraz yönetimi ve gerekli hukuki metinler ayrıca tamamlanır. Gerçek tutarla kontrollü uçtan uca kabul gerekir. Hukuki yükümlülükler faaliyet ve ülkeye göre ayrıca değerlendirilir; bu belge hukuki uygunluk onayı değildir.

Bu yol haritasının sonucu: “Daha çok teknoloji kullandım” yerine hangi hatayı engellediğimizi, hangi kapasiteyi hangi koşulda ölçtüğümüzü ve sorun çıktığında nasıl geri döneceğimizi gösterebileceğiz.

24. Karar özeti ve sık sorulan sorular

Başvuru için karar tablosu

Konu	Seçim	Gerekçe ve kabul edilen bedel
Mimari	Modular monolith	Yerel işlem doğruluğu ve düşük işletim maliyeti; modül sınırları testle korunacak.
Esas veri	PostgreSQL	Sipariş/stok/ödeme için tutarlı kayıt; kilit, sorgu ve yedek işletimi gerektirir.
Katalog cache'i	Redis + IDistributedCache	Tekrarlanan okumayı azaltma; TTL, invalidation ve ek bellek maliyeti.
Arama	Elasticsearch	Metin arama/facet; gecikmeli projeksiyon ve ayrı indeks işletimi.
Mesaj taşıma	RabbitMQ + MassTransit 8	Mevcut entegrasyonu sürdürme; teslimat/tekrar/versiyon kuralları yine gerekli.
Teslim güveni	Kalıcı outbox/inbox	Commit-yayın boşluğunu kapatma; ek tablo, worker ve izleme yükü.
Telemetri	OTel → Alloy → Grafana Cloud	Depoları VPS'te işletmemek; kota, ağ ve veri gizliliği yönetimi.
Yük üretimi	Ayrı makinede k6	HTTP kapasitesi ve tekrarlanabilir deney; browser deneyimini tek başına kapsamaz.
Yayın	Caddy + tek VPS + Compose	Bütçeye uygun ilk adım; yüksek erişilebilirlik sağlamaz.
İlk kapı	50 VU, 30 dk sandbox demo	Sınırlı ama kanıtlı başlangıç; 1000 kapasitesi ayrıca ölçülür.

“1000 kişi için Redis zorunlu mu?”

Hayır. Zorunlu olan beklenen yükte doğru ve yeterince hızlı davranmaktır. İndeksli sorgular hedefi sağlayabilir. Redis'i hangi tekrar yükünü azalttığını ölçerek ekleyeceğiz. Gereksiz cache bakım ve eski veri sorununu artırabilir.

“RabbitMQ varken outbox neden gerekiyor?”

RabbitMQ kendisine ulaşan mesajı taşıyabilir. Veritabanı commit'i ile mesajın RabbitMQ'ya ulaşması arasındaki boşluğu kendiliğinden kapatmaz. Outbox bu sorumluluğu aynı yerel transaction'da kaydeder. Tüketicide tekrar güvenliği yine gereklidir.

“Elasticsearch'te stok var; neden tekrar veritabanına bakıyoruz?”

Arama stok bilgisi gecikmeli bir görünüm olabilir. Satın alma kararı ise aynı son adede yarışan talepleri atomik olarak çözmelidir. Görüntüleme ile kesin rezervasyonun farklı kaynak sorumlulukları vardır.

“Monolith 1000 kullanıcıyı kaldıramaz mı?”

Mimari etiketi tek başına kapasite söylemez. Sorgu maliyeti, işlem süresi, kaynaklar ve kullanıcı davranışı belirleyicidir. Tek API süreci veya Worker sayısı ölçüme göre değişebilir. Mikroservise bölmek veritabanı yarışını veya yavaş sorguyu otomatik çözmez.

“Bütün işlemleri event'e çevirsek daha hızlı olmaz mı?”

Asenkron iş kullanıcı beklemesini azaltabilir ama sonucun gecikmesini, hatanın yönetimini ve durum takibini getirir. Sipariş kabulünden önce gereken stok kararı sırf hızlı cevap için ertelenmez. Arama projeksiyonu gibi gecikmeyi tolere eden işler uygun adaydır.

“Ödeme başarılı ekranı açıldıysa her şey doğru mu?”

Ekran yalnız sunucudan doğrulanmış durumu göstermelidir. URL parametresi kanıt değildir. Ödeme kaydı, sipariş durumu ve stok etkisi birlikte sınanmalıdır. Geç gelen başarı veya tekrar callback ayrı deneyler ister.

“Temiz ortamı doğrudan LoadTest yapabilir miyiz?”

Ancak kaynakları, anahtarları, veri temizliğini ve test adaptörü korumalarını buna göre ayırırsak. `clean` adı tek başına güvenlik sınırı değildir. Başka kullanıcıların denediği demo verisini test temizliğine dahil etmeyiz. Aynı makinedeki farklı veritabanı kaynak rekabetini de önlemez.

“İlk demo açılınca iş bitiyor mu?”

Hayır. Yayından sonra gerçek kullanım davranışı gözlenir, beklenmeyen yollar düzeltilir, yedek ve güncelleme disiplini sürer. 1000 VU ölçümü de yeni sürüm veya veri büyümesiyle yeniden değerlendirilebilir. Her kapasite raporu bir koşul ve sürüme aittir.

25. Terimler sözlüğü

Terim	Bu rehberdeki anlamı
Modular monolith	Tek ürün olarak dağıtılan, sorumlulukları modüllere ayrılmış uygulama.
Contract	Modülün diğerlerine açtığı arayüz, DTO veya olay sözleşmesi.
DbContext	EF Core'un bir çalışma kapsamındaki veri erişim/değişiklik takibi nesnesi.
Schema	PostgreSQL içinde tabloları ad alanıyla gruplama; ayrı sunucu değildir.
Migration	Veritabanı şema değişikliğinin sürümlü tarifi.
Seeder	Başlangıç veya test verisini hedef kaynağa yükleyen program.
Transaction	Belirlenen veritabanı değişikliklerini birlikte commit/rollback yapma sınırı.
Commit / rollback	İşlemi kalıcılaştırmak / o işlemdeki değişiklikleri geri almak.
İzolasyon	Eşzamanlı veritabanı işlemlerinin birbirinin değişikliklerini nasıl görebildiği.
Koşullu güncelleme	İş kuralını WHERE koşuluyla atomik olarak kontrol eden yazma.
Reservation	Ödeme beklerken belirli sipariş için ayrılmış stok.
Idempotency	Aynı iş niyetinin tekrarında ek iş etkisi üretmeme özelliği.
CheckoutAttempt	Belirli siparişe bağlı kalıcı ödeme başlatma/sonuç izleme kaydı.
Reconciliation / uzlaştırma	Sağlayıcı tahsilatı ile yerel sipariş/ödeme kayıtlarını eşleştirip farkı çözme.
Outbox	İş verisiyle aynı işlemde kaydedilmiş, teslim edilecek mesaj kaydı.
Inbox	Tüketicide mesajın işlenme durumunu tekrar kontrolüyle izleme.
ACK	Tüketicinin broker'a mesajı başarıyla aldığını/işlediğini bildirmesi.
Retry / redelivery	İşlemi yeniden deneme / mesajı daha sonra yeniden teslim etme.
Error queue	Normal işlemeyi geçemeyen mesajların incelenmek üzere ayrıldığı kuyruk.
Eventual consistency	Türetilmiş görünümlerin esas kayda gecikmeyle yaklaşması.
Projection	Esas veriden belirli okuma ihtiyacı için oluşturulan görünüm.
Cache-aside	Cache miss olduğunda uygulamanın asıl kaynaktan okuyup cache'i doldurması.
TTL	Cache kaydının tutulacağı süre sınırı.
Invalidation	Eski kopyayı geçersiz kılma.
Stampede / single-flight	Aynı miss'te çok sorgunun yığılması / aynı doldurma işini birleştirme.

Terim	Bu rehberdeki anlamı
Debounce	Art arda gelen kullanıcı girdisinde kısa sessizlikten sonra işlem başlatma.
Alias	Arama istemcisini fiziksel indeks adından ayıran mantıksal ad.
Log / metric / trace	Olay kaydı / sayısal seri / ilişkili teknik adımlar zinciri.
Span	Trace içindeki süreli bir işlem adımı.
Kardinalite	Bir metrik/log etiketinin ürettiği farklı değer kombinasyonu sayısı.
Sampling	Telemetrinin belirli bölümünü seçerek saklama.
OTLP / Alloy	OpenTelemetry taşıma protokolü / telemetri toplama ve yönlendirme bileşeni.
VU / iteration / RPS	Sanal kullanıcı / bir senaryo turu / saniyedeki HTTP istek sayısı.
Think time	Kullanıcının adımlar arasındaki düşünme/bekleme süresi.
p95 / p99	Ölçümlerin %95 / %99'unun altında kaldığı değer.
Throughput	Birim zamanda tamamlanan iş hacmi; HTTP RPS veya tamamlanan sipariş olarak tanımlanabilir.
Threshold	Testin otomatik geçme/kalma eşiği.
Backpressure	Kaynak sınırında yeni işi sınırlayarak aşırı birikmeyi önleme.
CI / CD	Otomatik doğrulama / doğrulanmış sürümün dağıtım süreci.
Liveness / readiness	Sürecin canlılığı / yeni iş almaya hazır oluşu.
Origin / CSRF	Protokol-alan-port birleşimi / tarayıcının oturumuyla istenmeyen istek yaptırma saldırısı.
HttpOnly / Secure	Cookie'nin JavaScript'ten okunmasını sınırlama / yalnız HTTPS ile gönderme.
Volume	Container'dan bağımsız kalıcı veri alanı; tek başına yedek değildir.
RPO / RTO	Hedeflenen veri kaybı penceresi / hizmeti geri getirme süresi.
RunId / manifest	Deneyi tanımlayan kimlik / veri ve koşulları tarif eden kayıt.

26. Kaynaklar ve belge doğrulama notları

Kaynak koduyla ilişki

Bu belge, 22 Ekim 2026'da `/Users/cagla/Desktop/TechCart` çalışma kopyasının kaynak incelemesiyle hazırlanmıştır. Aşağıdaki dosyalar mevcut davranışın örnek dayanaklarıdır. Dosyaların sonraki sürümlerde değişmesi belgeyi kendiliğinden güncellemez. Çalıştırma kanıtları Bölüm 1'de ayrı sınırlandırılmıştır.

- SDK seçimi: SDK 10.0.402 ve patch seçim politikası.
- Merkezi paket sürümleri: EF Core/Npgsql/MassTransit ve diğer paketler.
- API başlangıcı: yapılandırma, tüketiciler, kimlik doğrulama ve servis kaydı.
- Worker başlangıcı: ayrı süreç ve arka plan işleri.
- Sipariş oluşturma: mevcut çok adımlı kayıt akışı.
- Ödeme callback işleyicisi: sağlayıcı sonucu ve yerel iş etkileri.
- Temiz Compose kurulumu: ayrı kaynaklar ve bağlantılar.
- Kurulum notları: önceki çalışmanın kurulum adımları ve profilleri.

Yerel dosya bağlantıları bu bilgisayardaki çalışma kopyasını gösterir. Belge başka makineye taşındığında proje kökü değişebilir; dosya adları rehberin inceleme haritası olarak kullanılabilir. Gizli ayar dosyaları içerikleriyle birlikte belgeye alınmamıştır.

Resmî teknik başvuru listesi

Kaynaklar 22 Ekim 2026'da kontrol edilen çevrimiçi belgelerdir. Sürüm, fiyat ve hizmet kotası gibi bilgiler uygulama/yayın zamanında tekrar kontrol edilir. Bu rehberin kapasite hedefleri kaynakların verdiği TechCart garantileri değildir; projeye ait kabul kararlarıdır.

- EF Core transaction ve context paylaşımı
- ASP.NET Core yapılandırması
- ASP.NET Core dağıtık cache
- ASP.NET Core antiforgery
- MassTransit outbox kavramları
- MassTransit EF 8.5.7 paketi
- MassTransit lisans koşulları
- OpenTelemetry .NET
- OpenTelemetry log korelasyonu
- Grafana Cloud OTLP ve Alloy mimarisi
- Grafana Cloud güncel kotaları ve fiyatları
- PostgreSQL 17 pg_stat_statements
- PostgreSQL 17 EXPLAIN
- Npgsql bağlantı ve havuz parametreleri
- Elasticsearch alias yönetimi
- iyzico Checkout Form sonuç sorgulama
- k6 açık ve kapalı yük modeli

- k6 geliş hızı executor'ı
- k6 execution ve VU kimlikleri
- k6 SharedArray ve fixture belleği
- k6 thresholds
- k6 test türleri
- Testcontainers .NET
- Playwright
- Caddy otomatik HTTPS
- GitHub Actions .NET
- PostgreSQL 17 yedekleme
- PostgreSQL 17 pg_dump
- PostgreSQL 17 pg_restore

Belgenin kullanım ve güncelleme kuralı

Bir bölümü uygularken önce mevcut kaynak tekrar okunur, sonra dar kapsamlı değişiklik yapılır ve ilgili kanıt üretilir. Bu rehberdeki C#/SQL/k6 örnekleri tasarım açıklamasıdır; açıkça belirtilen mevcut endpoint örneği dahi bu çalışma kapsamında çalıştırılmış değildir. Hedef örneklerin dosya veya endpoint'i henüz bulunmayabilir. Yeni sonuç geldiğinde “planlanıyor” ifadesi ancak o sonucu gösteren kayıtla değiştirilir.

Markdown, PDF'nin içerik kaynağıdır. PDF'deki kapak, içindekiler sayfa numaraları ve sayfa başlıkları sunum katmanıdır; esas metin, tablolar, örnekler ve kaynaklar aynıdır. Bu teslimde içerik eşleşmesi ve sayfa düzeni kontrol edilir. Uygulama kodu, veritabanı ve çalışan servisler bu belge üretim işi kapsamında değiştirilmez.